

Announcement for CSE 134 Alumni

Looking for participants for research on the benefits of coding assistants

- Requires 2.5 hours in person at CSE
- You will:
 - Complete programming tasks using our VSCode-based platform.
 - Take a few surveys.
- Eligibility requirements:
 - **At least 3 months of JavaScript experience.**
 - **No prior experience with React.**
- To register: Scan the QR code or email: shj002@ucsd.edu
- Compensation: \$50 gift cards

Sorry new folks



If you have any questions, please let us know.

Shaokang Jiang (shj002@ucsd.edu)

Michael Coblenz (mcoblentz@ucsd.edu)

```
//CSE  
110;
```

Design Process Basics

Brainstorming, User Centered Thinking and Requirements Gathering

with Thomas Powell

```
//CSE  
110;
```

Welcome to Level 2: We Broke The Predictability Illusion

Programming Assignments (PAs) are obviously challenging, especially if you are new to learning how to code, but they may often create an illusion of predictability in emerging dev's minds.

 If you do experience such predictability beyond school it is in fact an illusion or you are working on a very simple or mechanical problem (ex: a port). Likely in this case someone else is taking care of things for you and may hide the challenges they face. This control buffers remove your actual control which can be quite frustrating as you may not be able to influence outcomes greatly.

The Truth About Hand Me The Spec



Mark Dalglish @markdalglish · Feb 24, 2019

I don't understand developers who say they don't care about design and just want designers to hand them a spec.

Bad design leads to bad code.

💬 52

↻ 428

❤️ 1.7K



If we did go this way...

Then our job become assembly or more code manufacture and we believe that the code is the complexity rather than code solution that maps to the problem being the hard part.

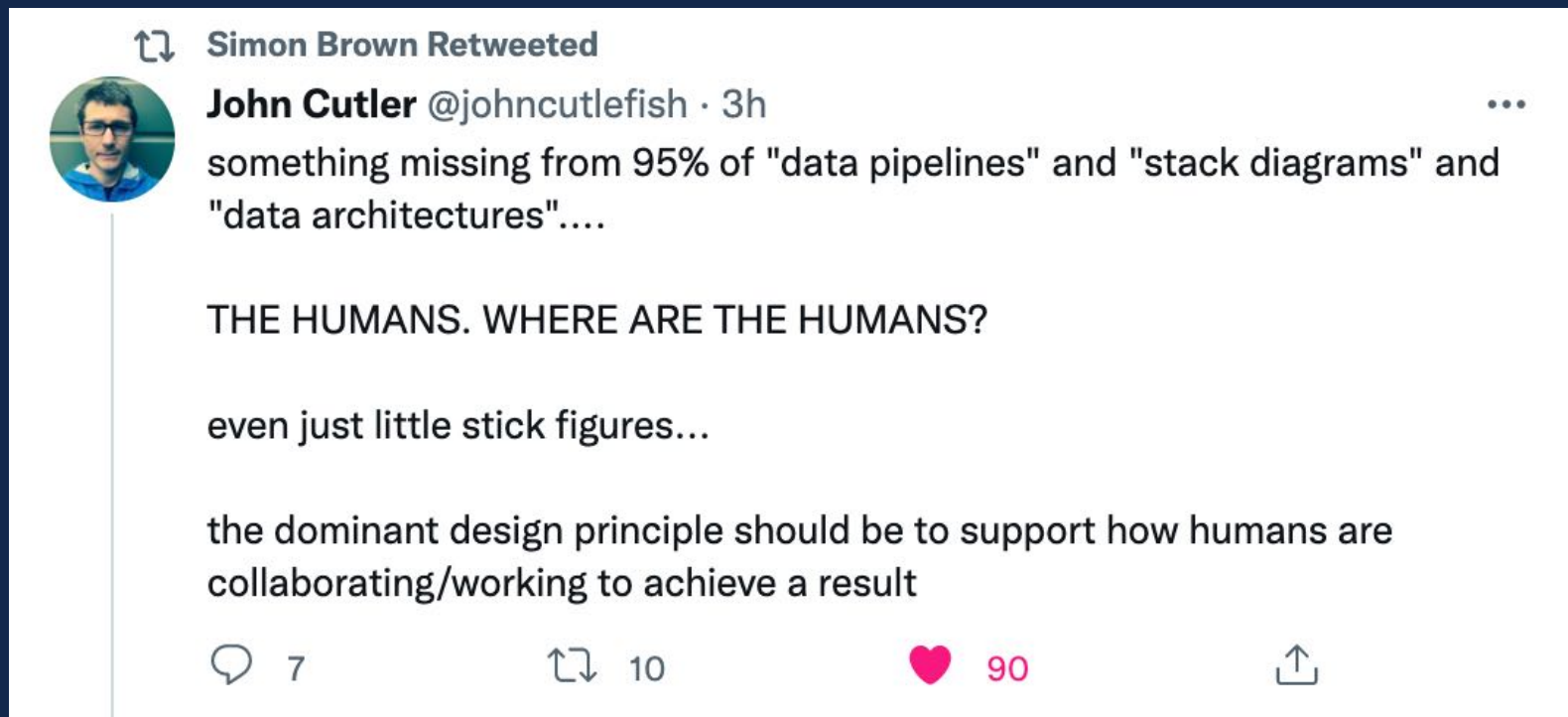
Worry: Manufacture seems less a valued skill and what can be defined precisely is executed by cheaper less skilled labor or even automated. (ex. GPT-4?)

Who then does that design and spec? If such analysts are the common place where are they coming from? Are these people devs? Where is our formal spec style we all adhere to ?

Worry: Per my experience this is absolutely not what is going on and BAs are becoming rarer and rarer and glib statements of direction being used as requirements leaving many decisions up to interpretation. Hard to do engineering like that!

Even if a “hand the spec world” could exist it would need to address the subjectivity in the system - the people/users to be a realistic spec.

Nobody Lives in Our Code Utopias?



The truth as I understand it is that even a little user knowledge and appropriate design thinking is the ultimate secret weapon of a high performing software engineer. Sure it is subjective but....

It runs into this idea which I certainly have found to true(ish)



Philip Bump  @pbump · Aug 14, 2018

People only want objectivity to the extent that it conforms with their subjectivity.

 10

 33

 155



Are We Just Coders ?

OR

Are We Complete Problem Solvers?

Which Comes First?

The coding or the problem solving?



Jack Domleo @jackdomleo7 · Aug 1

Software developers & engineers are problem solvers who write code.

Not coders who solve problems. 🤔

💬 68

↻ 586

❤️ 2.9K





Mads Brodt @madsbrodt · Aug 1



When we work as developers, we're not paid to code. We're not paid to use different languages and frameworks.

We're paid to solve problems – for businesses and users.

Code just happens to be an efficient tool to reach that goal. But don't lose sight of the real objective.



46



722

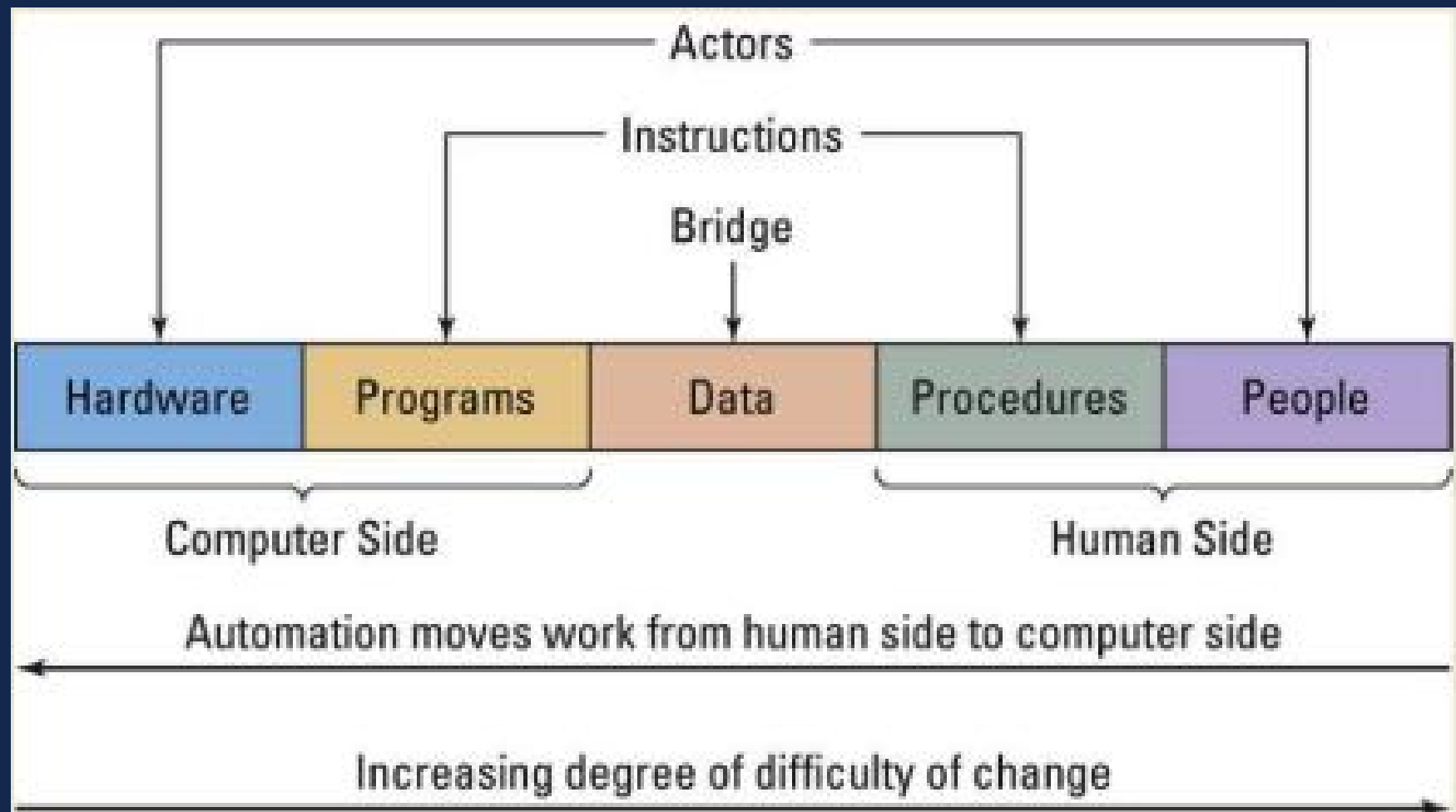


2.8K



[Show this thread](#)

Always Think of Systems - SE, Product, etc.



Ok Then What to Solve?

(Specific) What vs (Specific) How

The challenge of figuring out specifically what to code is arguably much more difficult than solving the how to code problem.



Danger: Avoid straw men and hasty generalizations to rationalize any code supremacy bias you might have upon hearing this widely accepted thought.

*“A general statement of “Solve AI” is not a “specific what,” but a general one. This type of requirement deserves a general how of “Write Code” or “Use ML” as a response. It isn’t necessarily a bad **prompt**, but it isn’t really a detailed enough requirement to work with.”*

Broad Prompts vs Specific Requirements

- The previous statement might seem debatable if you confuse vision, prompt or goal with a specific requirement
- For example, “Solve AI” as a prompt is quite easy to declare, but if 50+ years of effort is clearly quite hard to accomplish.
- Prompts are purposefully general and broad and not enough information to implement from. Prompts serve as a point of departure nothing more.

Prompts Are Important

A prompt helps us succinctly know what our goals are and the rough direction we are going with a project.

Not all prompts are good ones:

- ✗ Bad prompts aren't well thought out and simply might not be reasonable to accomplish given the situation or constraints
- ✓ Good prompts have been thought about and should rely on significant experience and data that suggest they are appropriate.



Probe prompting stakeholders for reasoning and details you may be unaware of

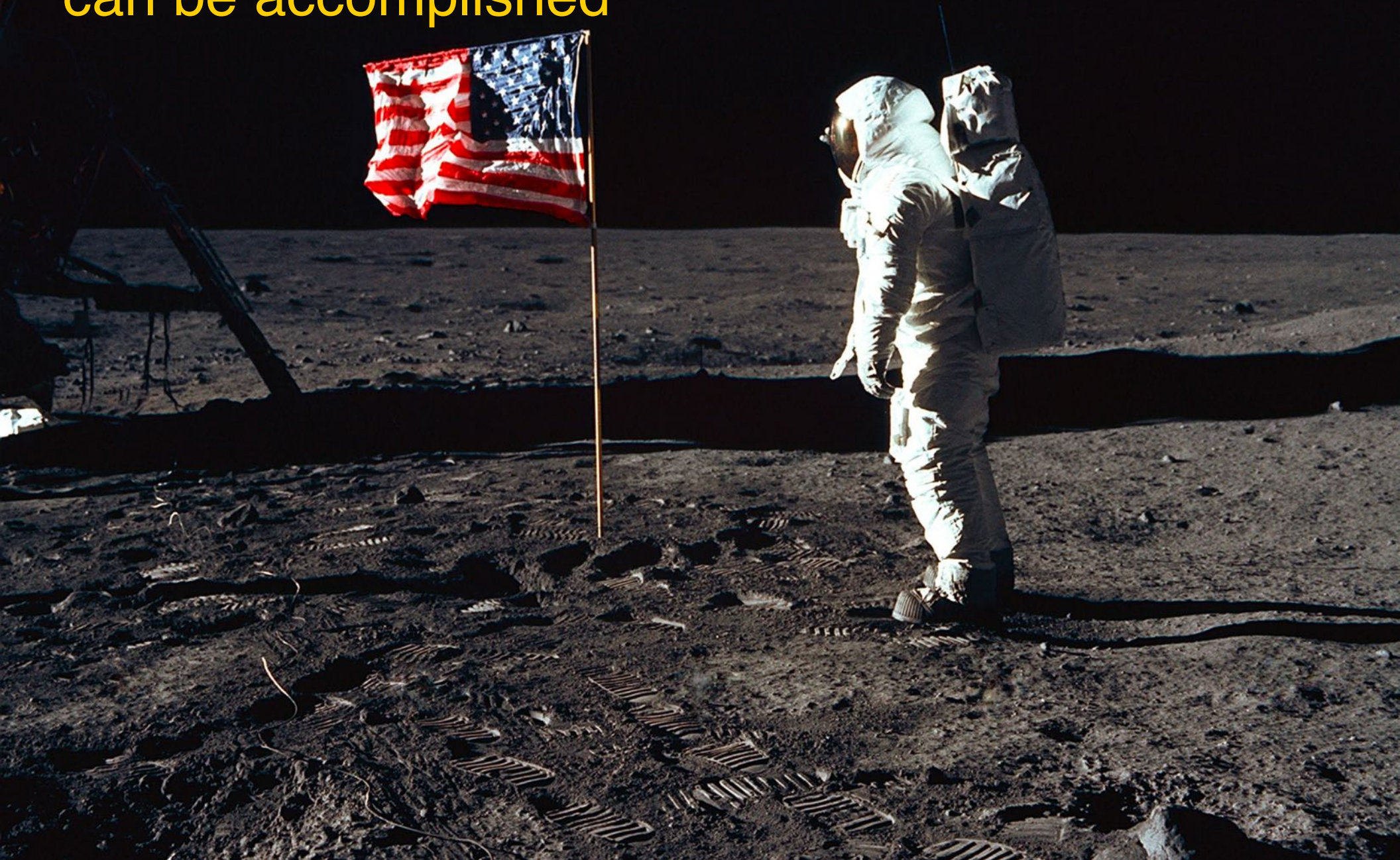
The Prompt By Another Name is Still the Prompt

The first prerequisite you need to fulfill before beginning construction is a clear statement of the problem that the system is supposed to solve. This is sometimes called "product vision," "vision statement," "mission statement," or "product definition."

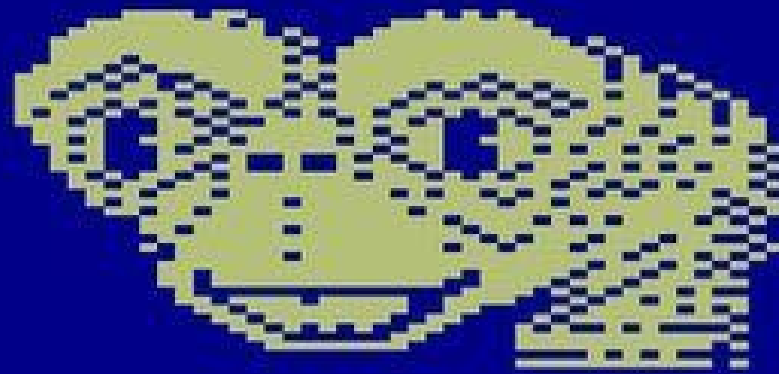
Code Complete
Steve McConnell



Prompts are powerful. Given enough time,
money and perseverance just about anything
can be accomplished



...but should you address a prompt should not be a given



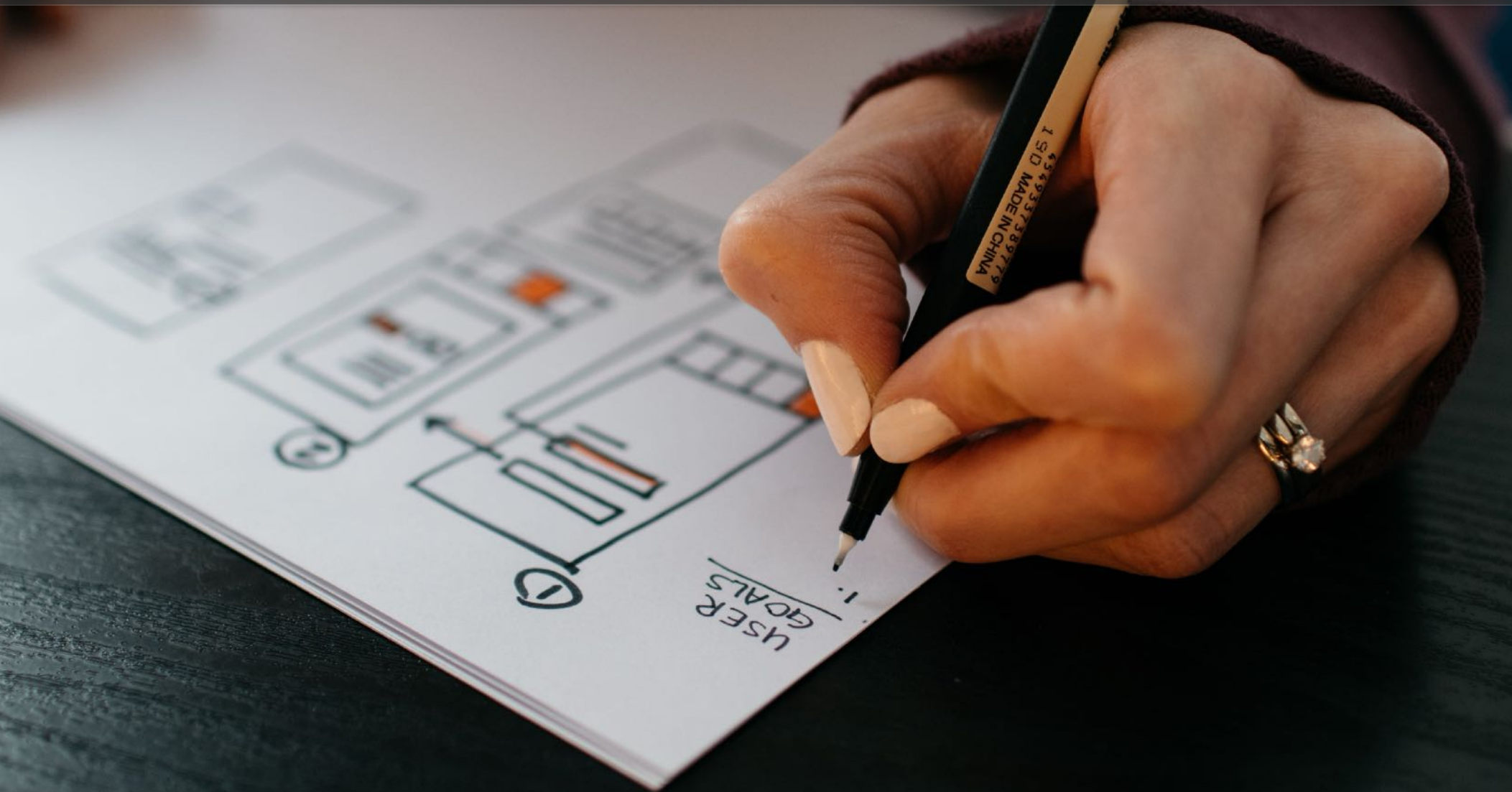
Assuming a good prompt
are we ready?

Prompt. Code. Fail.

For Lamport, a major reason today's software is so full of bugs is that programmers jump straight into writing code. "Architects draw detailed plans before a brick is laid or a nail is hammered," he wrote in an article. "But few programmers write even a rough sketch of what their programs will do before they start coding." Programmers are drawn to the nitty-gritty of coding because code is what makes programs go; spending time on anything else can seem like a distraction.

The Coming Software Apocalypse
The Atlantic

Planning is Much Cheaper Than Coding



We Must Figure Out Specifically What To Do?

- Normally someone else prompts you and depending upon situation may even provide you some degree of requirements.
 - In a start-up you may even be involved in the prompting.
- This idea that someone else prompts and others respond to meet the prompt isn't foreign to most disciplines.
- Again SE isn't really that unique and I suggest we think of our "closest cousins" for some guidance.



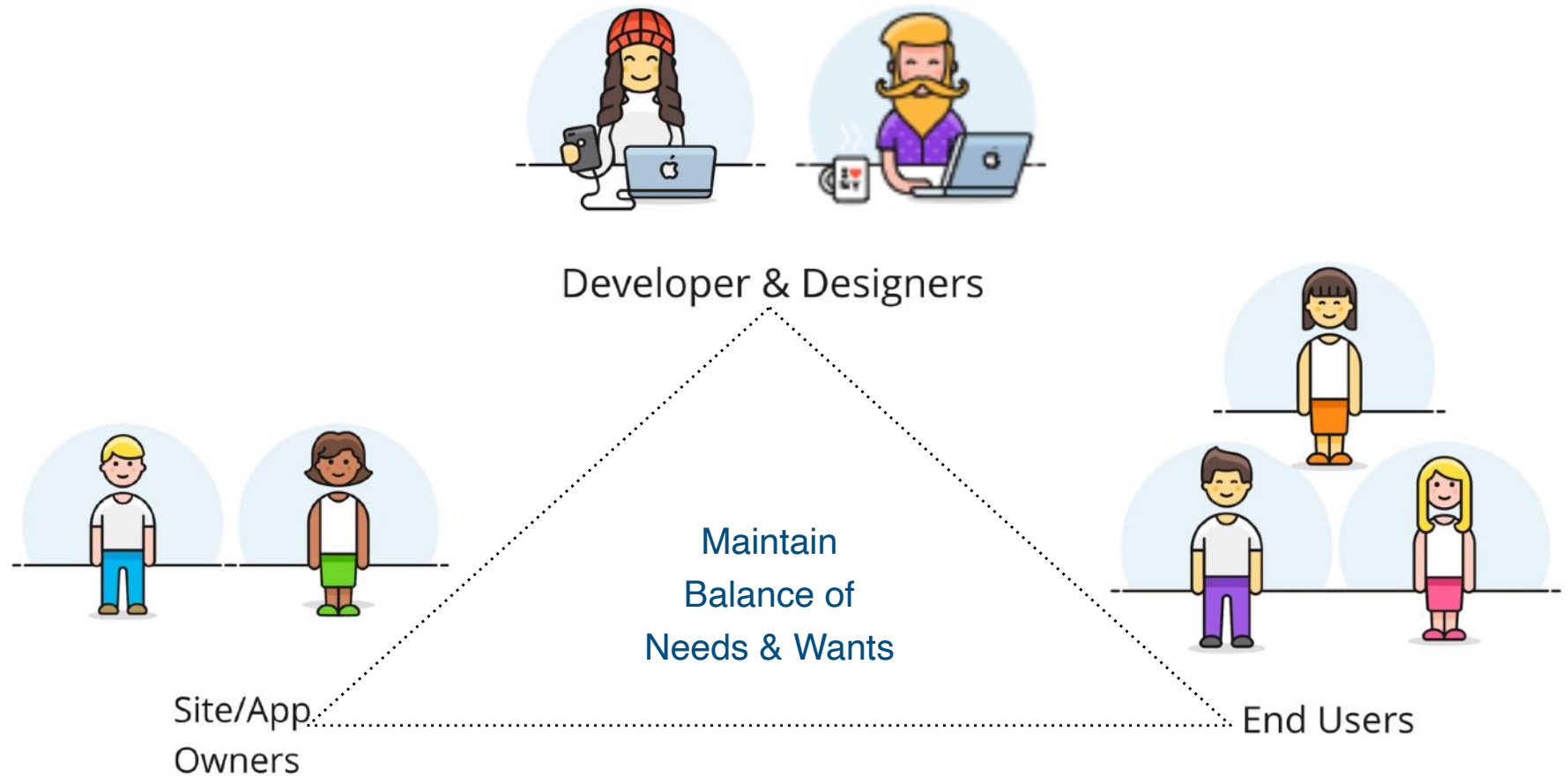
SEs and Architects

Our closest professional cousins marry engineering and design. They also suffer the same type of “fashion trends,” customer foibles, variability of execution, intersection of disciplines, and challenges of trade-offs.

Exploration of Analogy

- Good architects interact with and consider the users of their work
- Good architects address trade-offs
- Good architects respect materials and process
- Good architects are often on site participating and guiding the effort as it goes
- Good architects fully consider the impact of their work on people and the “environment”
- If we replace architects with dev, SE, designer, etc. it should still hold.

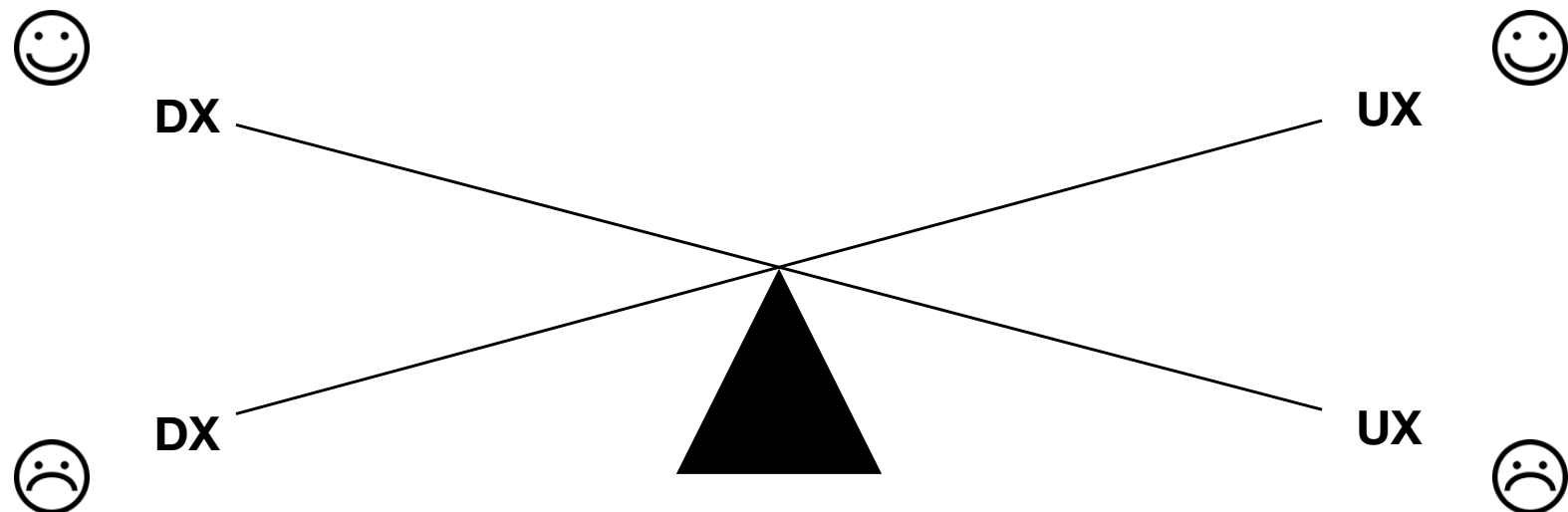
We Maintain the Balance



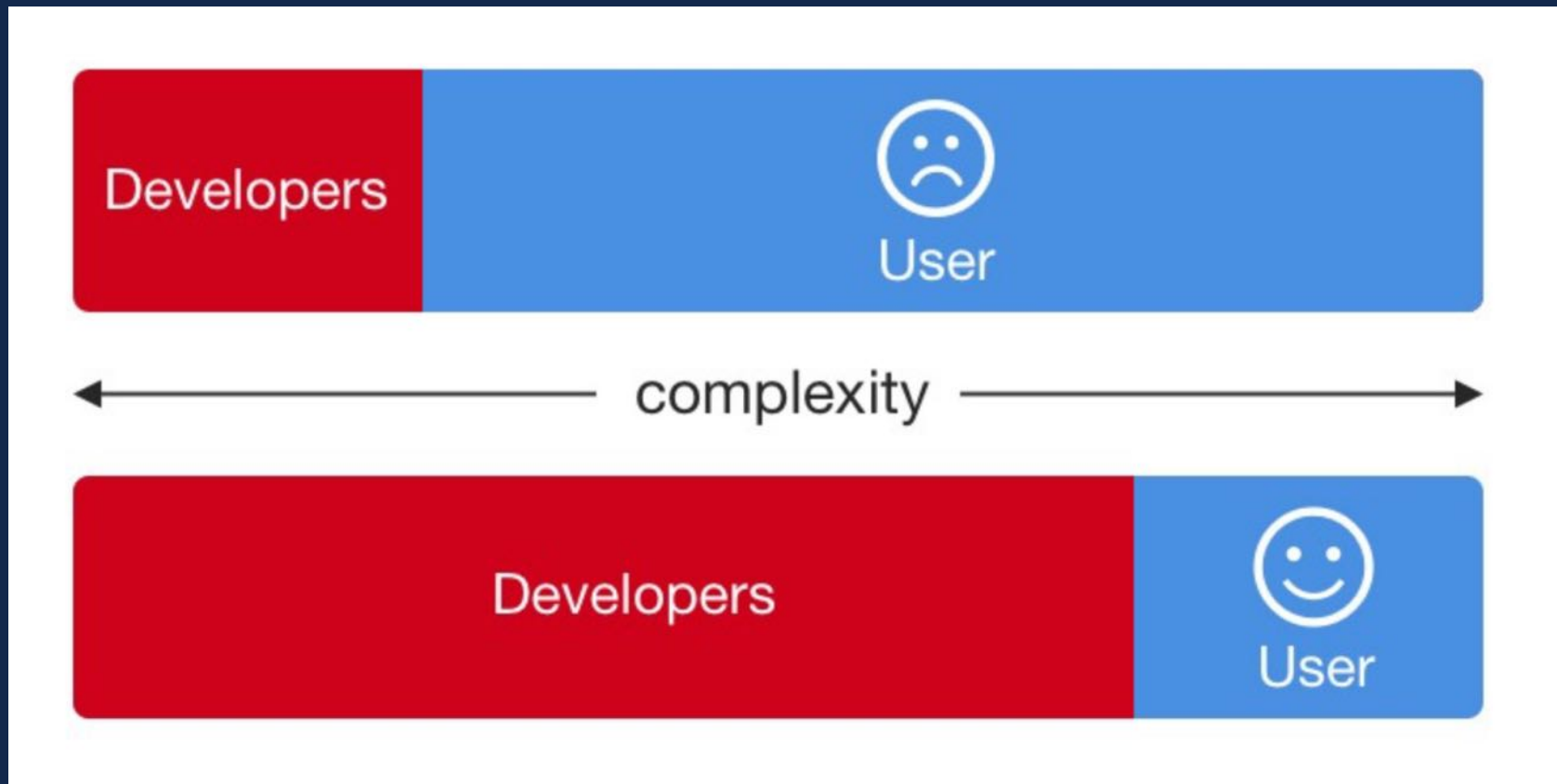
Balance Example

Developer Experience High

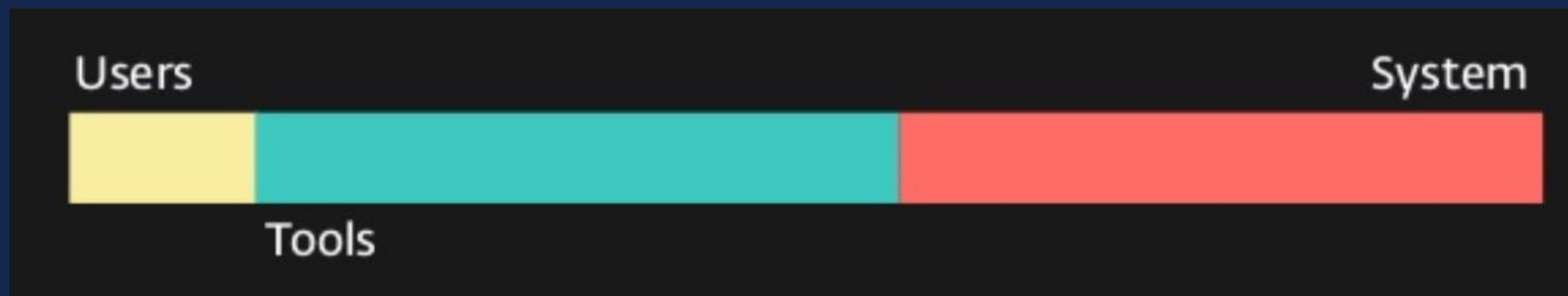
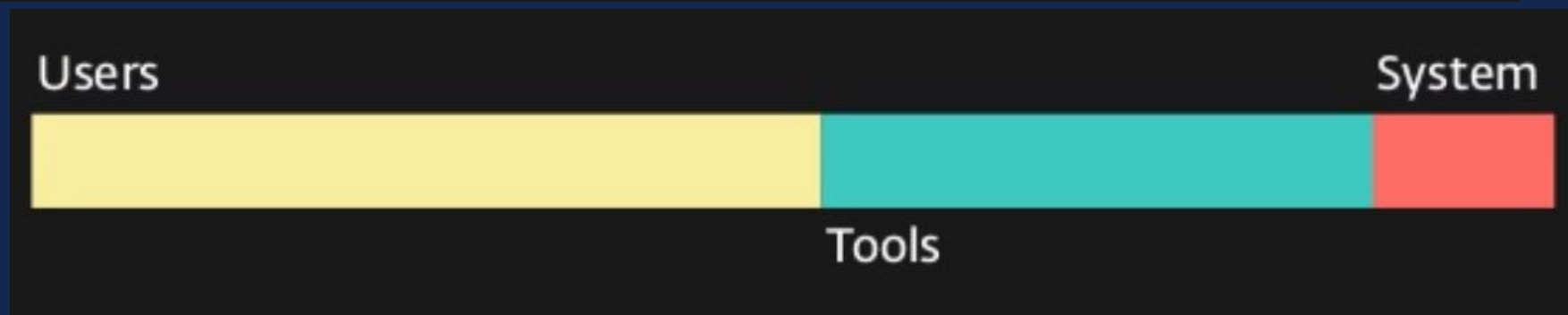
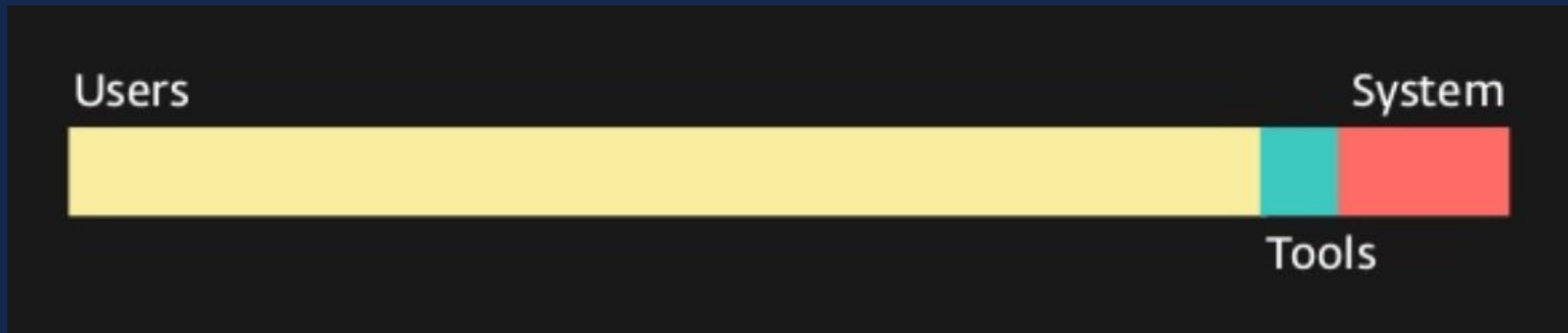
often leads to performance details
result in better performance and UX



Another Version

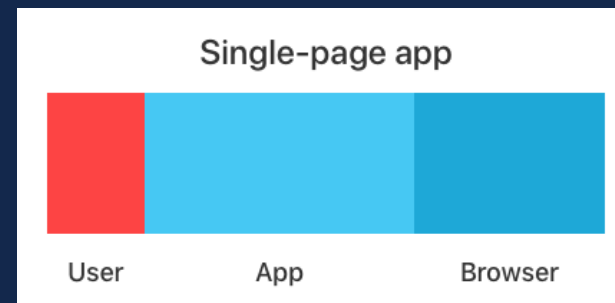
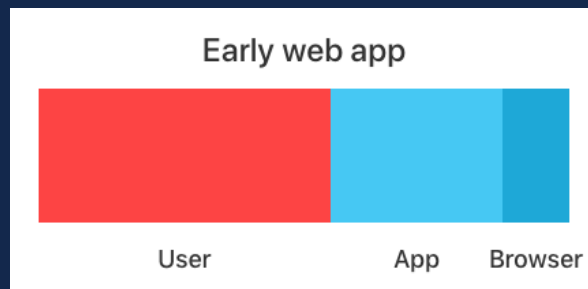


This Style of Trade-Off Abounds



Tesler's Law

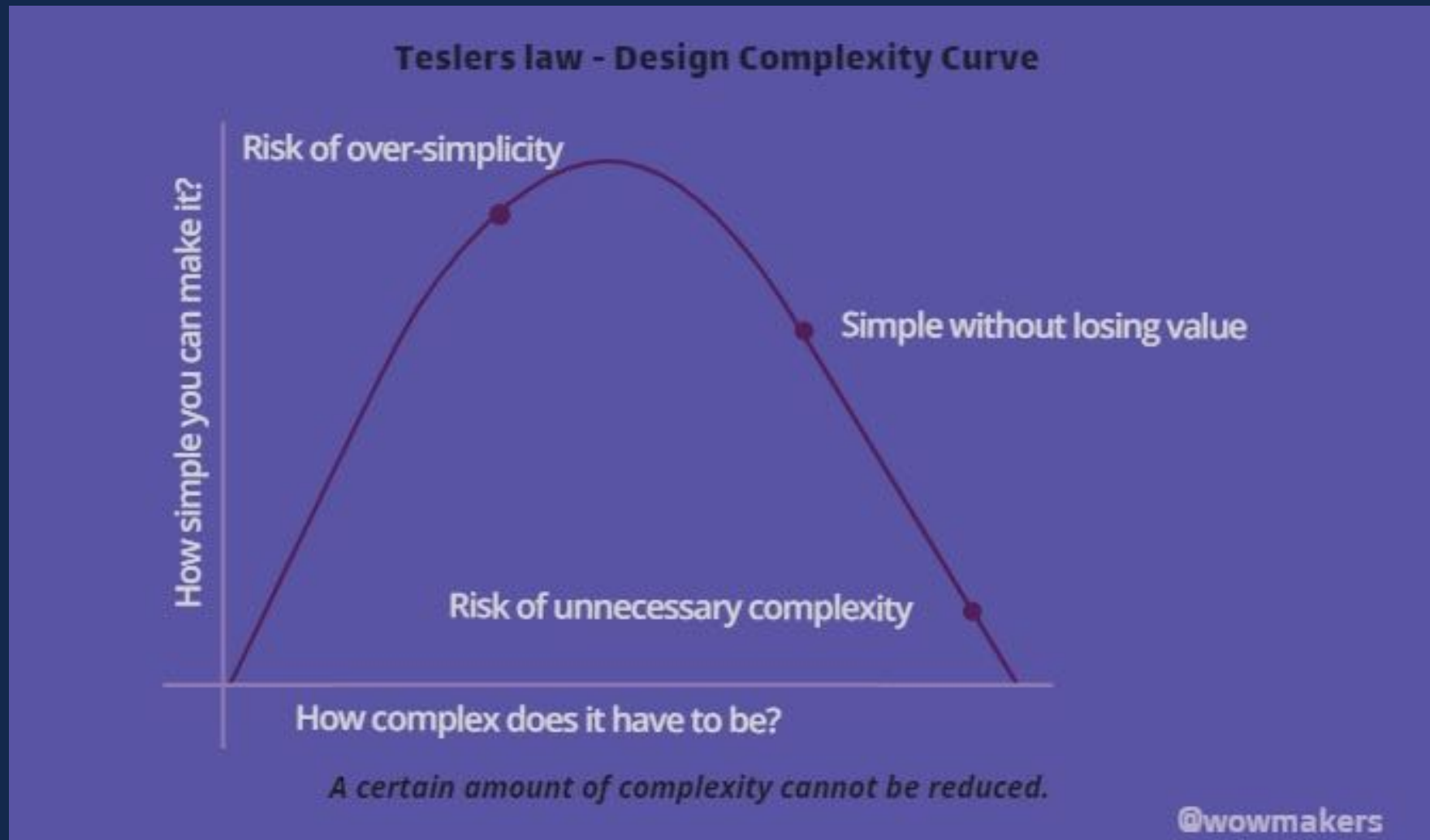
- A conservation of complexity law states that the inherent complexity of something can't be reduced beyond a certain point. Once at that point you just shift where the complexity is placed.



Shift the burden around, but the general complexity is preserved

https://en.wikipedia.org/wiki/Law_of_conservation_of_complexity

Tesler's Law



DON'T BE KYLO THE CODER



MORE

Requirements Elicitation

- Requirements elicitation is the practice of researching and discovering the requirements of a project from users, stakeholders and other involved parties.
 - At my firms we generally call the process Discovery
- Gathering and discovering requirements can be difficult. Reasons include:
 - People often do not really know exactly what they want
 - Communication and shared understanding can be difficult
 - The appropriate process of eliciting responses varies widely by people and the problem faced
 - The organization and distillation of varied discovery content can be messy and time consuming

Beware of HiPPOs



Tame Your HiPPO

- Highest Paid Person's Opinion (HiPPO)
- Just like the dangerous hippo of the wild, these hippos will eat your work
- Make sure your Hippos believe in data and core software concept and want to accept the results
- Be careful though HiPPOs often do have wisdom, you may need to give room to that too!



Watch the Requirements Pit

Requirements are not just lying around to be picked up the often have to be mined with much labor from beneath layers of assumptions, misconceptions, and politics

Sometimes requirements can't be mined they have to be synthesized

The real world is messy, conflicted and unknown

Exact specifications are rare, and sometimes impossible

The job of devs is help people to understand what they want

Sadly few people know exactly what they want



Guard Against Over-specification

Go Goldilocks

The opposite of too little requirements is too much

Watch out for volatility of requirements like “policy” and situational reqs

Remember requirements are not architecture or design then drive those

Stake Holder Elicitation

- Talking to stake holders about what their wants are is generally a process of interviews and on-going Q&A.
- Stake holders may often need to be guided in the discovery process and can become confused or frustrated with technically or overly detailed solutions focused discussions.
- Stake holders may attempt to assert a dominance in the process that SEs must **tactfully** reject the same way a Doctor squashes a WebMD “expert.” Be careful here you are talking to bosses/customers/etc. but if you don’t do this you are potentially doomed before you break code ground.



Charles Lambdin @CGLambdin · Mar 10



Agile teams that build whatever users say they need are sort of like doctors that prescribe whatever meds their patients ask for.

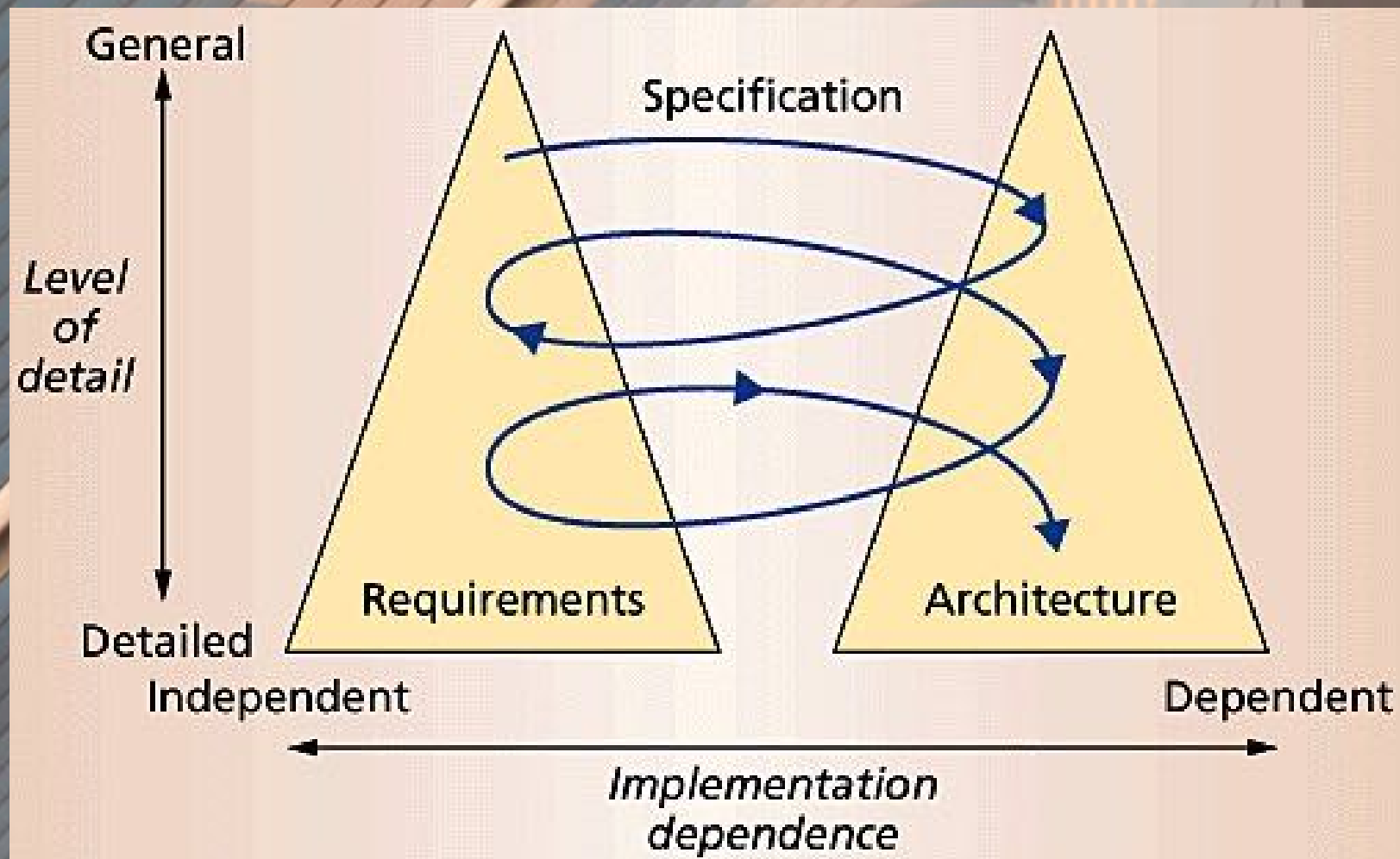


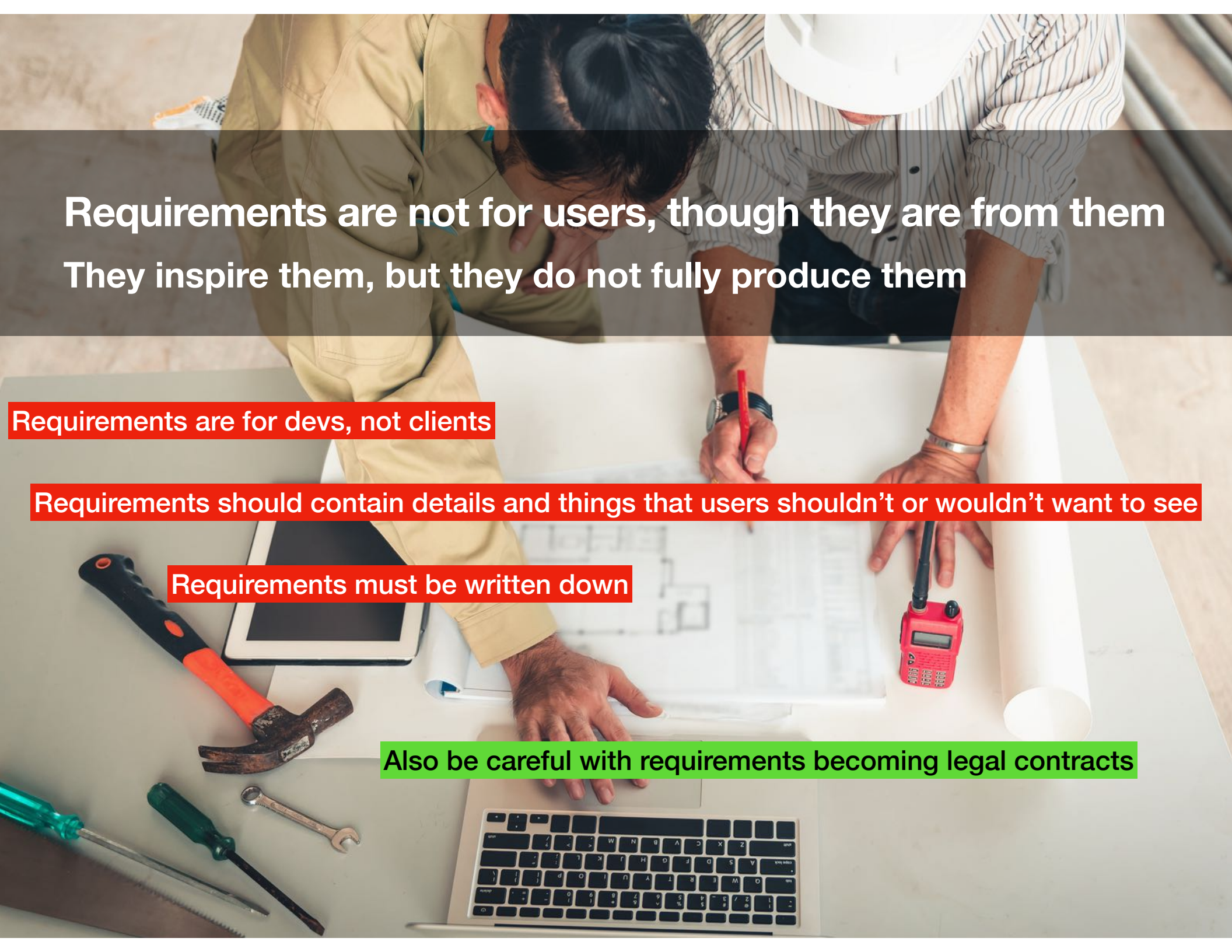
User Requirement Discovery

- Talking to users is different than stake-holders, they will have different thoughts than your patrons
- Be careful making users into designers
- Be very careful not to spook them and attempt to employ UCD in some limited fashion given your time / project constraints. We'll talk about that in a moment

Requirements, Specification and Architecture

Usually Not Completely Different, Typically Iterated





Requirements are not for users, though they are from them
They inspire them, but they do not fully produce them

Requirements are for devs, not clients

Requirements should contain details and things that users shouldn't or wouldn't want to see

Requirements must be written down

Also be careful with requirements becoming legal contracts

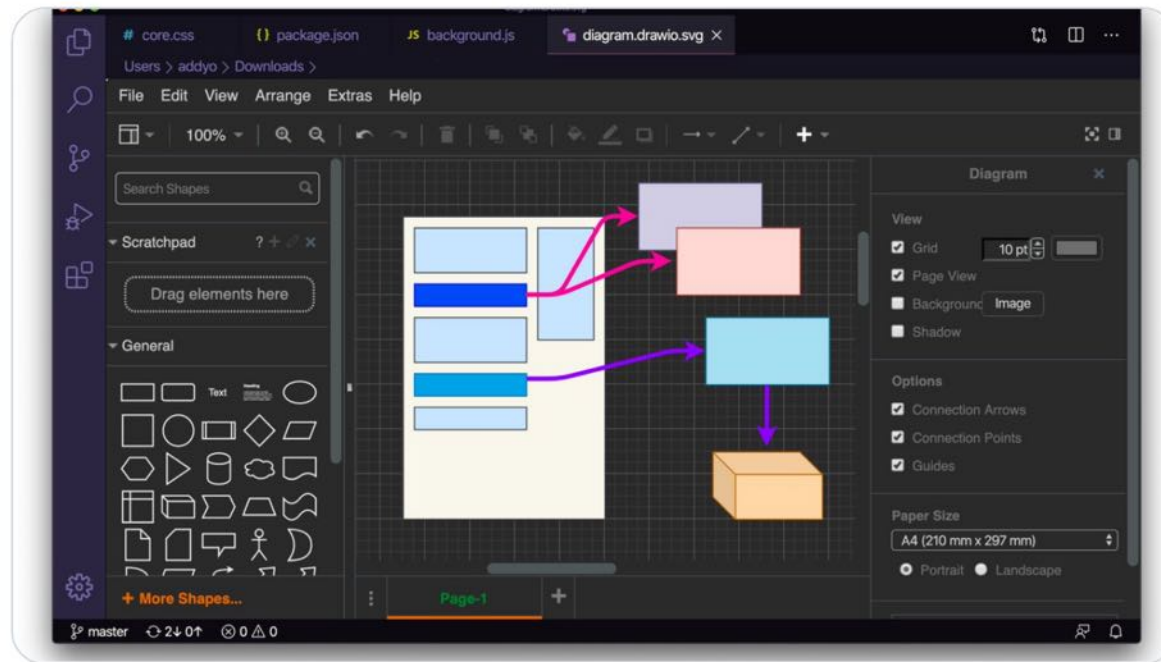
Why this excites me

Because maybe we should collapse the distance between thinking, planning, and coding



Addy Osmani ✓ @addyosmani · Jun 17

Wow. You can create and edit full diagrams in your editor with the [Draw.io](#) VS Code extension: bit.ly/draw-vsc



45

931

3.4K



UCD Overview

User Centered Design (UCD)

UCD means we keep end users top of mind when we create things for them

Remember there are always things for us as we celebrate our craft, our tools, our industry, etc. Remember, we are operate at a different level. We must see the software as the user sees it.

The Primary Law of UCD

Creator $\neq$ Consumer

Differing needs

Differing wants

Differing understanding

Differing mindsets

They are usually different people

...and that's ok

It's always about the end users

- Users declare us good or bad for better or worse
- They often only see results and things above the water
- Call this “The Iceberg Model”
- Treat these people well as after this class they are the ones that grade us!

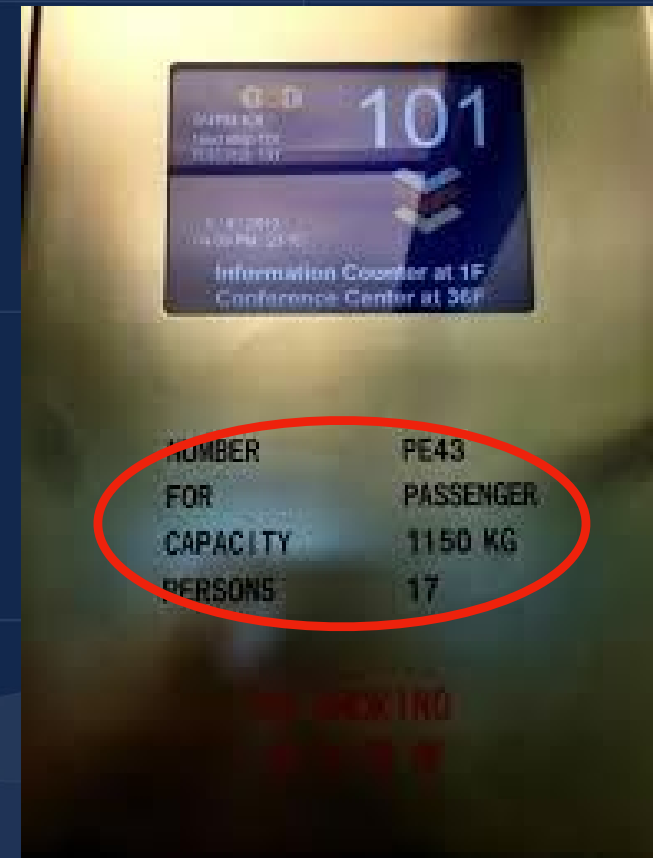


Solve for...

- When we are trying to problem solve we find that constants/literal values (aka constraints) make solving the equation easy
- Example: $4x + 5 = 25$ Solve for x .
- The variable x has one value only (5) our problem is quite solvable.
- What about adding more variables potentially less constraints?
 - $4x + 5y = 25$. Many solutions, infinite if you allow reals
 - $4x + 5y = 10z$
 - $x + y + a + b + c + \dots = z$

Math Life Problems

- Life isn't Math but the constraints of math formulas are useful to us to consider
- Better with Constraints Scenario
 - Boss - Build an elevator
 - You - Ok, how much kilograms? How many people?
 - Boss - A lot, I don't know maybe 10 people.
 - Choice: Get to work or get more specific?



Engineering Constraints

Definition: *constraint* - a limitation or restriction

Then no constraints aka “unconstrained” means without limits

“**Realistic** engineering requires accepting **real** constraints.”

“Engineering without accepting constraints is just winging it and is inherently risky.”

Avoid Sharpie Constraints!



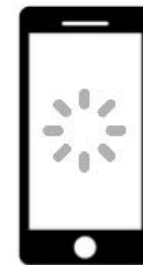
Human Constraints

- At a base level human constraints bound what is reasonable for us to develop towards
- We have two primary constraints categories to consider
 - Senses
 - Brain
- I avoid cognition and just say “brain” because I think we start to get into other related areas like interest and attention as well
- Other constraints exist, but are less physical and more situational

User Capabilities

- While there may not be a totally typical person, people do have similar capabilities. Remember the main way that a user interacts with our programs likely employs:
 - Thinking and Memory
 - Movement and Reaction time => time awareness
 - Sight
 - Other possibilities may include hearing

RAIL Constraint



Response

100ms

Animation

60fps

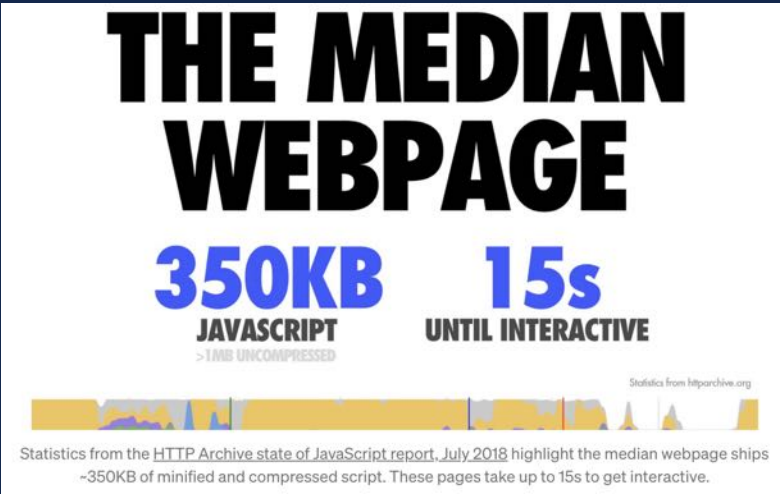
Idle

50ms

Load

1s*

*We are starting to address this as meaningful paint or use metric and other more precise metrics like time to interactive as opposed to first contentful paint are often used. Also given delivery constraints this bound may be adjusted but we don't have to be loose and Google is pushing a performance budget as both an SEO signal as well as something to console warn about.



If client-side JavaScript isn't benefiting the user-experience, ask yourself if it's really necessary. Maybe server-side-rendered HTML would actually be faster. Consider limiting the use of client-side frameworks to pages that absolutely require them. Server-rendering and client-rendering are a disaster if done poorly.

CSS
JS THE MOST EXPENSIVE
IMG PART OF YOUR SITE.
FONT

Yet



Acknowledge a Maximalist Trend Also Present

- Some psych here in regards to subtractive solutions and how students are aghast about why I de-emphasize additive solutions often widely promoted ones.
- Many reasons ranging from dependency reduction/fragility/longevity, attempting to get you to avoid heuristic based solutions until you understand constraints (ex. Wisdom of crowd downsides), encouraging you to be a broader thinker, becoming comfortable with cognitive dissonance and finally acknowledging broad imperfection and subjective variability in dev decisions you will face as early as possible in your career



Pro Tip: Senior devs and management will likely tell you many “bad things” about something (tech, pattern, process, etc.) and yet still use it, encourage you to use it, etc. You should be asking them why they say that not coming up with stories to how this doesn’t follow. It will reveal deeper thinking and compromise awareness.

Context Matters

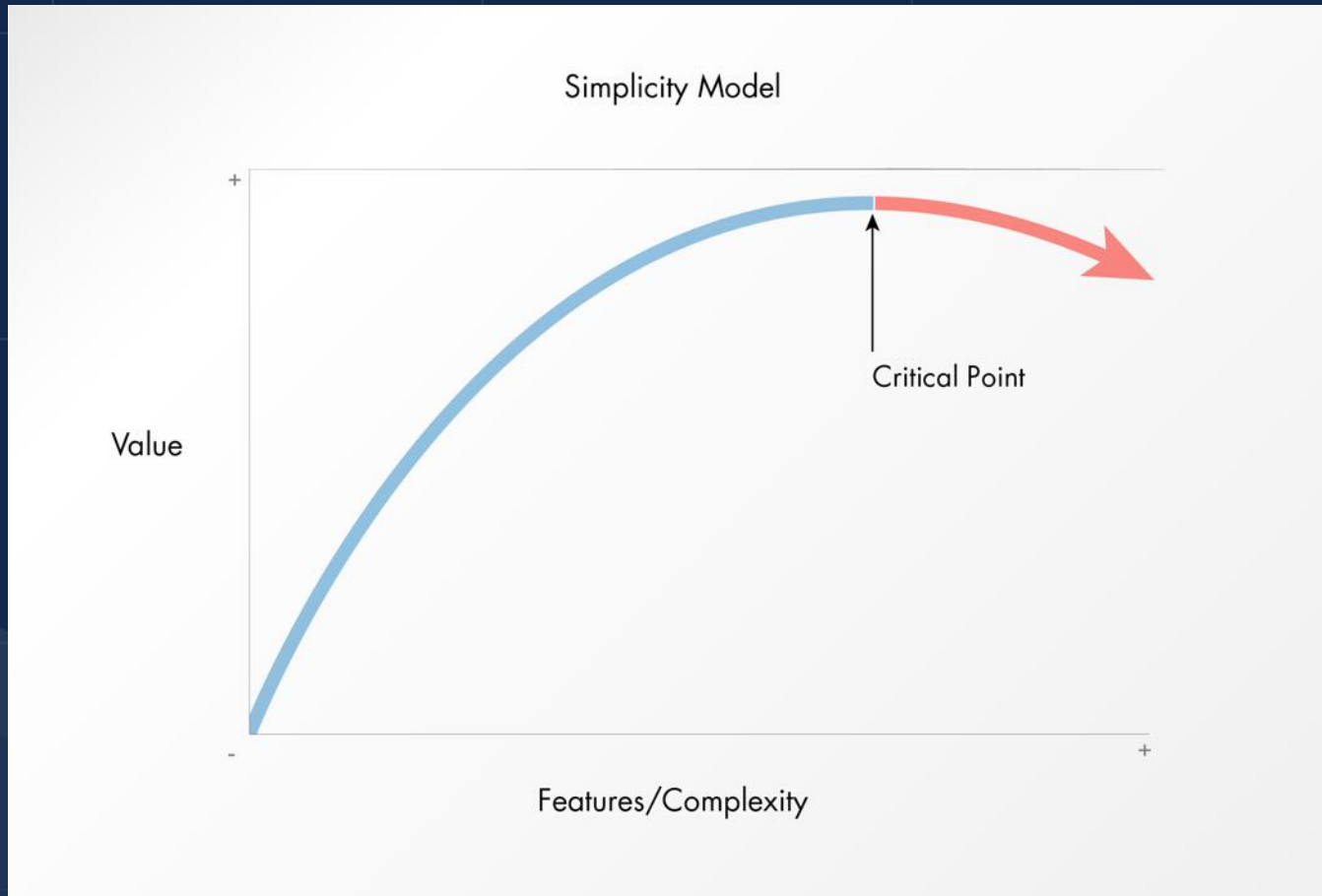
- We also see that environmental and situation concerns can significantly modify a user's interpretation of our software
- Put another way context really really matters to users
 - Ex: Mobile vs Desktop, Public vs Private, Reasoning why user is employing the app
- Context also includes culture and experience.
 - Ex: Amazon effect and my 99% rule idea

Example

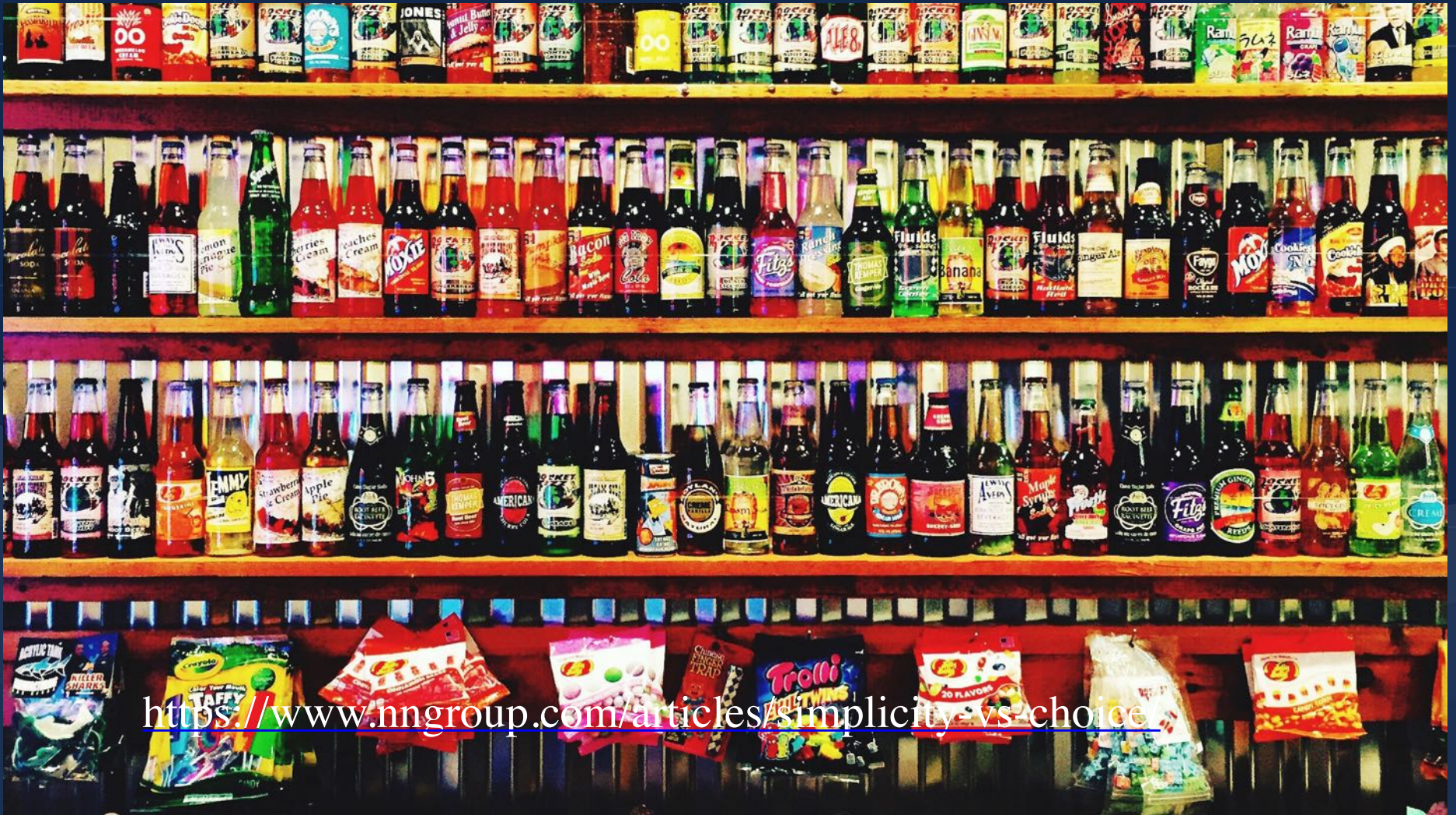
In general the number of choices issues and ability to deal with stimuli suggests that indeed “less is more” when it comes to a piece of software, but why then do we often have more?

- Comparison Heuristics
- Economics
- Loss Aversion

Feature / Complexity Curve



Wealth or Tyranny of Choice?



<https://www.nngroup.com/articles/simplicity-vs-choice/>

Trade-offs - Deal with Them

As we introduce constraints since we humans certainly have them we are going to have situations where there isn't a clear best of everything solution.

Marketers present that you can have both and no compromises are required.

Is that really true? **NO!**

Extreme Danger Warning: Devs view themselves more logical than they actually are about this situation. See current Webdev trends if you need any proof.

The Miller Lite Paradox

Inherent Assumption: A choice is required



Promise: You can have both

Reality: At best you might compromise on both

Respect The 99% Rule

People are mostly not consuming and using only the software we make

In other words 99% people are elsewhere and being influenced by what they see/do there and with other apps

Familiarize yourself with the conventions seen on the common 99% and adopt those (with consideration) to improve usability

Yet Avoid Death By Similarity

We conform to style guides and conventions slavishly with just subtle variations we can fail

When everything is the same nothing is important

Apply a “seasoning” rule for design. Add seasoning to bring some “zing” to your app, just don’t over do it!



Be the same, but different!?

Being the same as what is commonly found elsewhere can get boring for users

Being the same doesn't mean you have to be exactly and predictably the same, you can be somewhat better or different for

As example your team's app shouldn't look and act just like everything else you see in this course and beyond. It should have its own personality and attention to detail so user's recognize it as different, but not too different

Think of Actual Users

- General categories and characteristics is a good place to start but understanding YOUR particular audience is much better
- Fact: The actual audience varies from project to project and you must ponder or even study your audience BEFORE making a piece of software



Observe Users in the Wild

- If you can you should go meet users to really understand what they want out of your application
 - Meet them in their actual environment, not a lab
 - Don't make them do your job (Users != Designers)
 - “Walk in their shoes” if you can
 - And if possible observe passively and try not to spook the users
 - Be careful of the test effect!
- Given our remote situation and time constraints this isn't going to happen. However, this is not excuse to give up the exercise entirely

We must enter the user's world



...and help guide them in ours

Artifacts Ahead

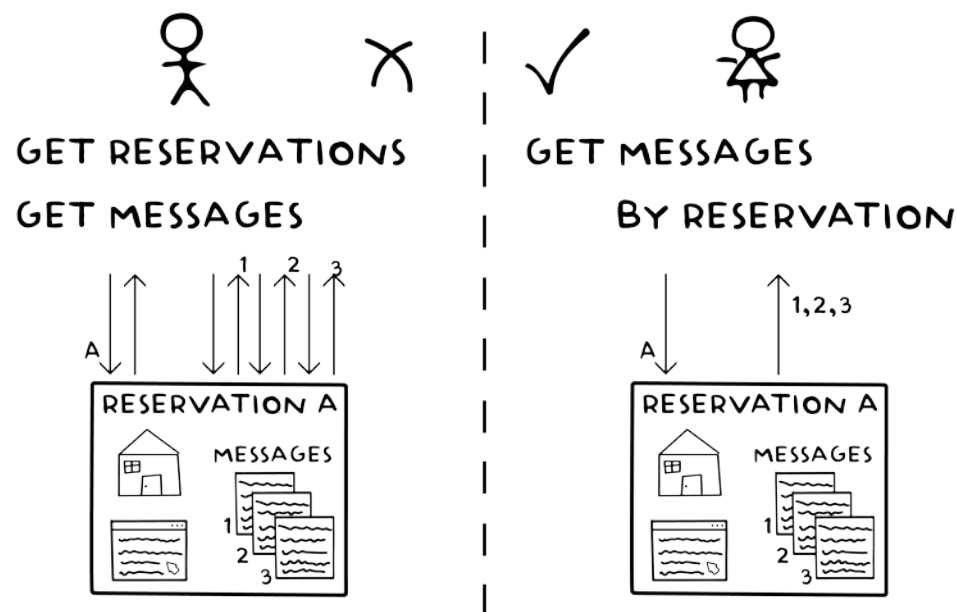
- For now we are doing high-level and conceptual thinking about the topic of UCD, users, etc.
- Artifacts for our project are previewed now
 - Diagrams describing broad flows
 - Mood and idea boards
 - Personas
 - User Stories
 - Journey or Flow Maps
 - Fatmarker Sketches
 - Wireframes
 - High Fidelity Comps and Prototypes
 - Also plenty of system and data diagrams too! ... After you know what you are building and for whom though!

Aside for those not buying
user focus / “design”
thinking ideas so far

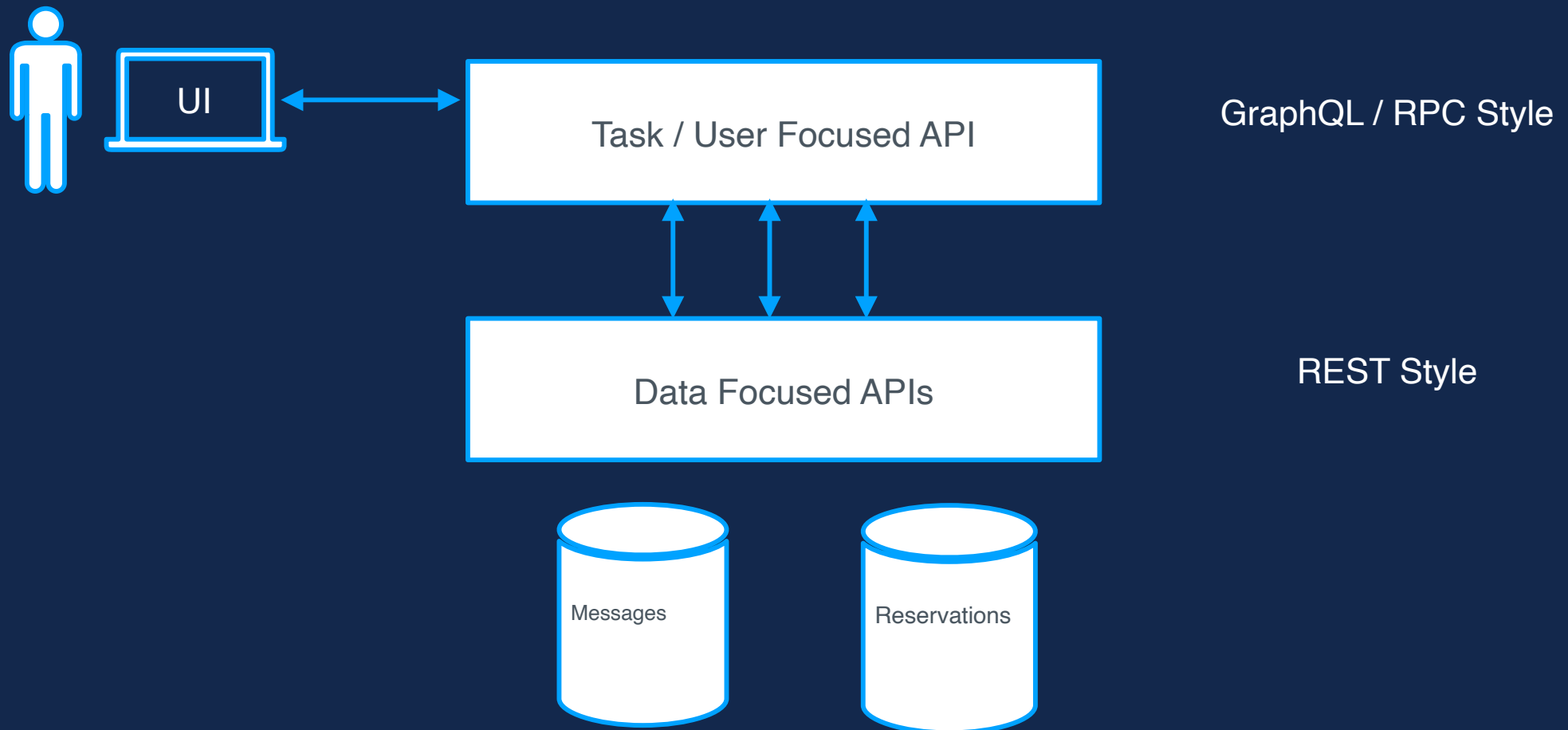
APIs Should Be User Focused?

- Often times we build APIs from a data point of view
 - This can work but it might result in more calls or not be actually contextual
- Consider instead thinking about your APIs from a user point of view
- Downside - changing user requirements
 - Solution - Layer between the public API (task/user focused) and internal API (data focused)

BUILD CUSTOMER CENTRIC DOMAIN APIs



Users + Data Thinking for the Win?



Again we do this for users not computers. If you reject this I assume it will catch up to you... eventually.

Back to thinking about
users...which don't always
have it easy by the way

Never Forget Accessibility

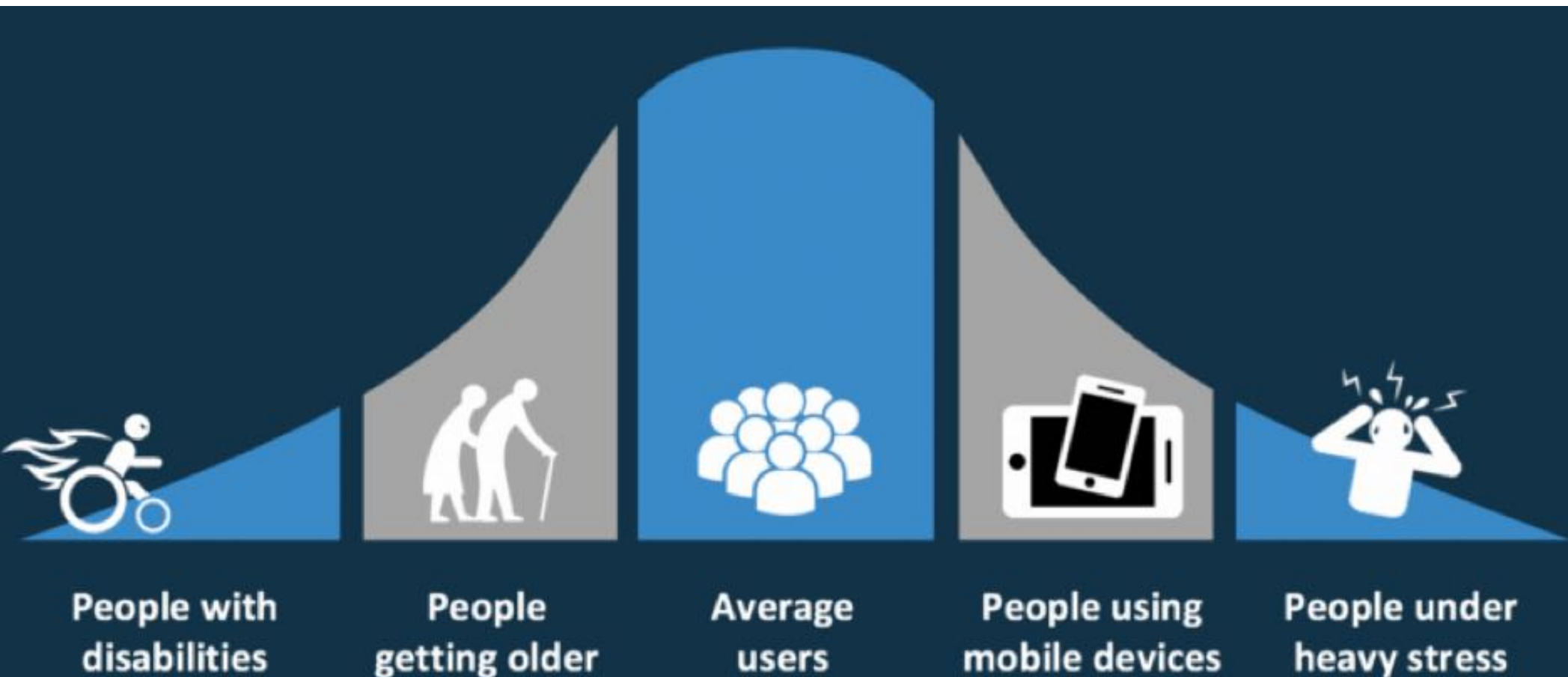


Accessibility

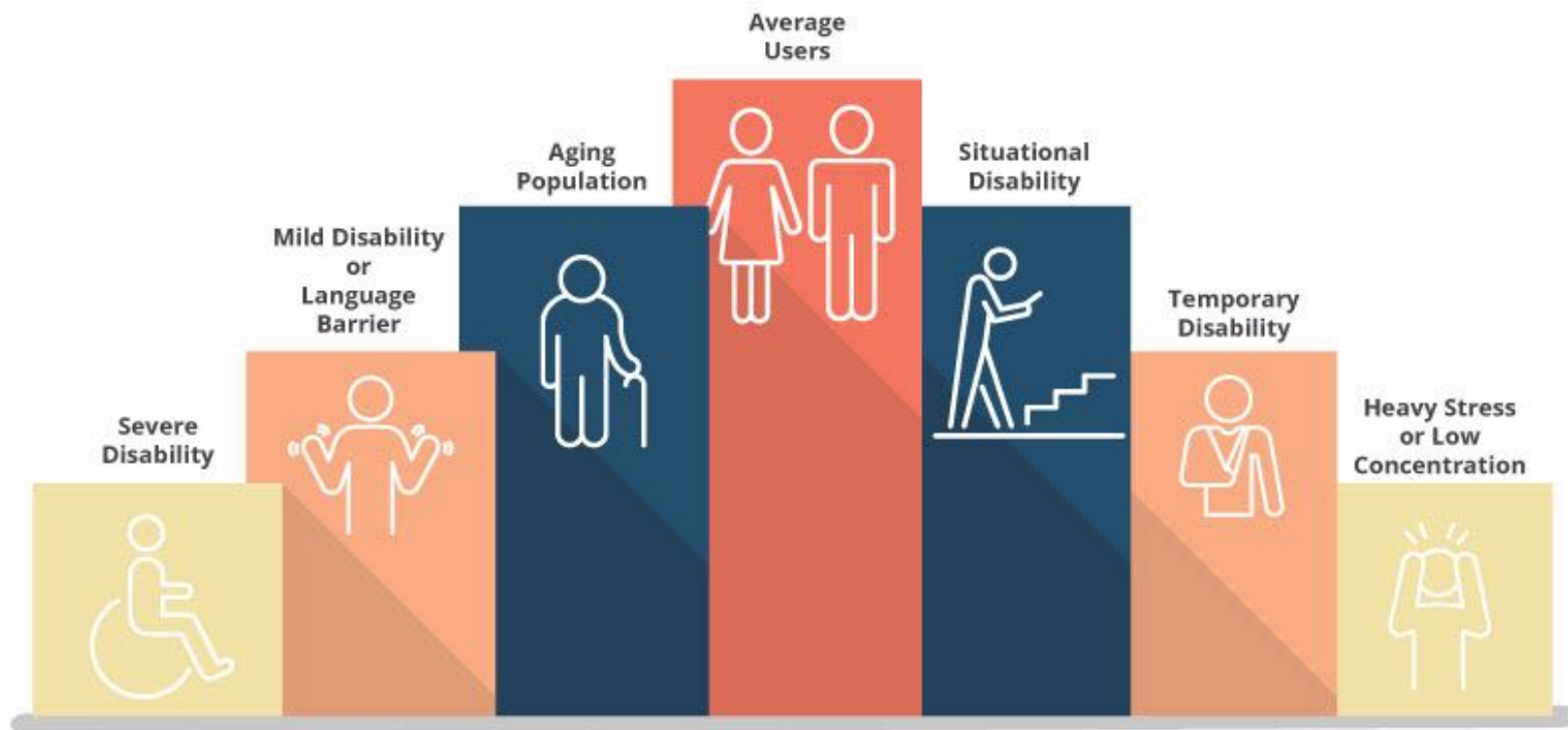
- What to do about the limitations of real users including the disabled? - make something accessible
 - Vision, movement, hearing constraints
 - Reading and comprehension constraints
 - Less than ideal access environment and technology
 - Accessibility might also address those who are not well off equipment wise for other reasons
 - Today we are seeing how technical privilege is locking people out of important content they need for health and well being

TL;DR - Consider well the possible ranges of accessibility

Accessibility Range #1



Accessibility Range #2



Great (User) Expectations

- Even with perfect accessibility the user really is the most difficult constraint we must deal with
- Couple this challenge that generally over time, user expectations grow while patience lessens
- Reasonable or not, end users want lots of features as fast, visually pleasing and potentially even entertained by what you build

Ok we got it we always need to
be thinking of users as we do
this so now let's figure this out...

Brainstorming Time

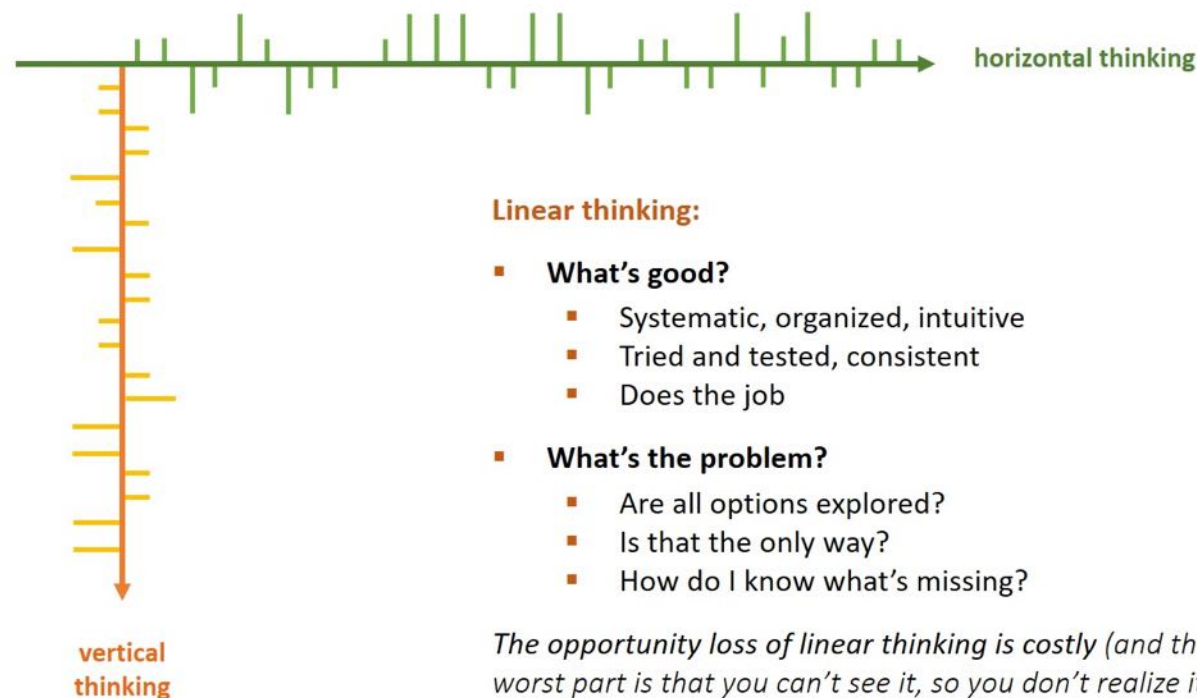


First off being creative by
yourself is difficult



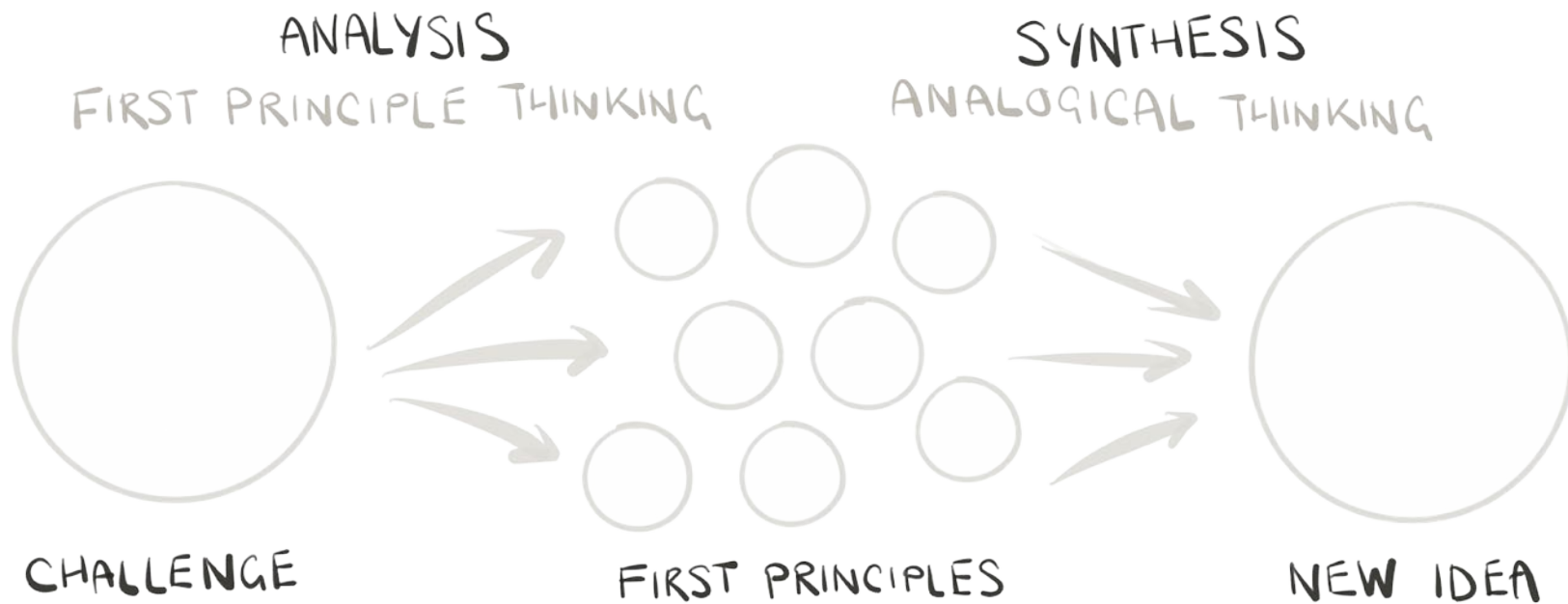
Tip: Think Both Directions

Linear thinking is our default: We are very good at **planning** and **making lists**



Pro tip: People typically do either list making or planning, and not both. By doing both, i.e. list making then planning, we're already one step closer to lateral thinking.

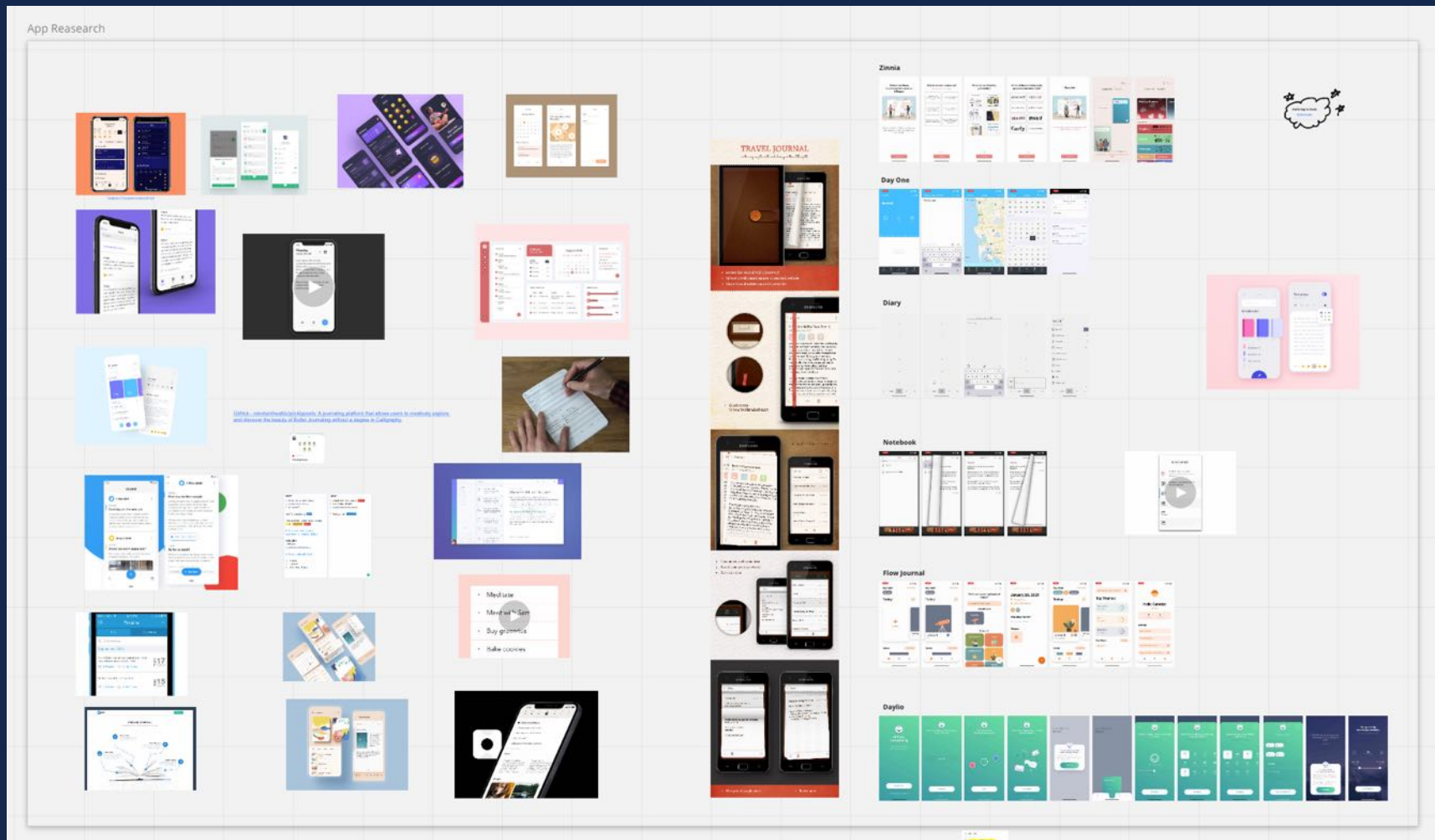
THINKING FROM FIRST PRINCIPLES



Research Phase

- Deep dive on what's out there and available already
 - What other apps are on the market for what you are trying to do / are adjacent to you?
 - What do those apps do well? What do they fail at? What are they missing
 - What do the reviews for those apps tell you about what users want? (This does NOT count as user research - live user research is critical)
- This research *is not* to give you a blueprint of what to copy, but to give possible ideas for some features / what to implement

Research Phase



Consider - The Domain

Thinking About the Domain

- Ours is fortune teller app, but I can't in good faith do that app here as I will steal your design thinking
- I will show you instead some simple thoughts on domains from previous quarters that did pomodoros, bullet journals, etc.

Take 1 - Bullet Journal

Thinking About the Domain

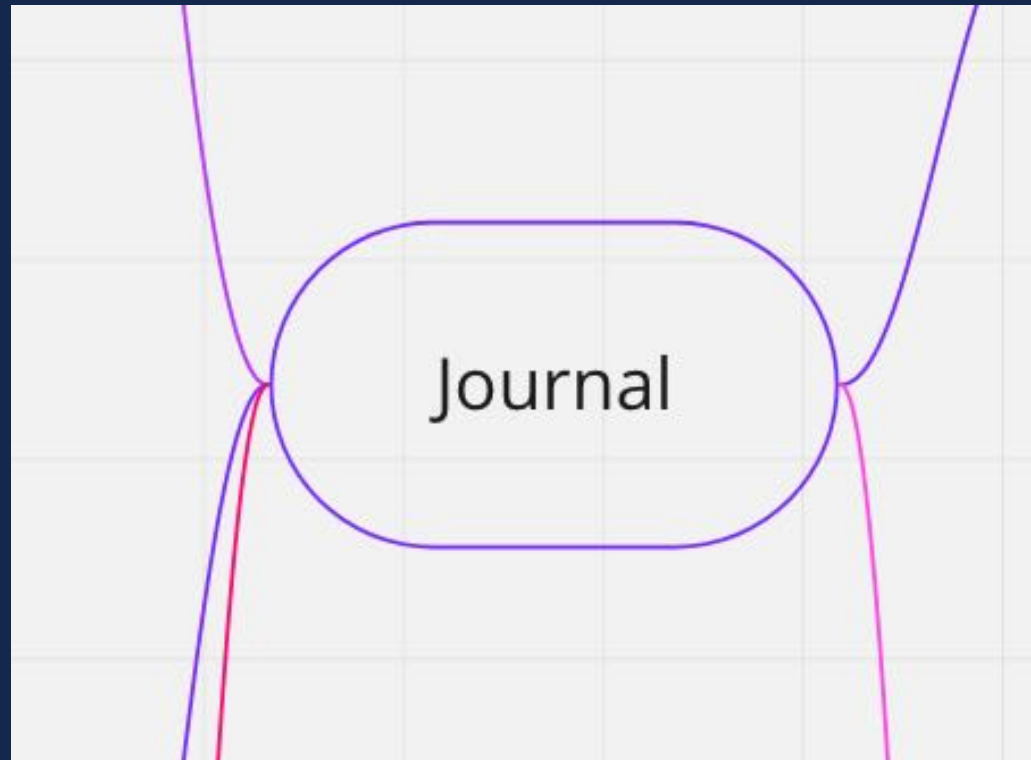
- It's a bit arrogant to assume that just because you've done something before you've already considered everything about that domain and don't need to critically analyze it
- Going into making a bullet journal, we had assumed that it was just a journal, simple CRUD app, the domain wasn't very deep
- Quickly we realized the rabbit hole went much deeper than we thought. Example: Should one store your diary online in someone's DB? Seems dangerous!!!

Thinking About the Domain

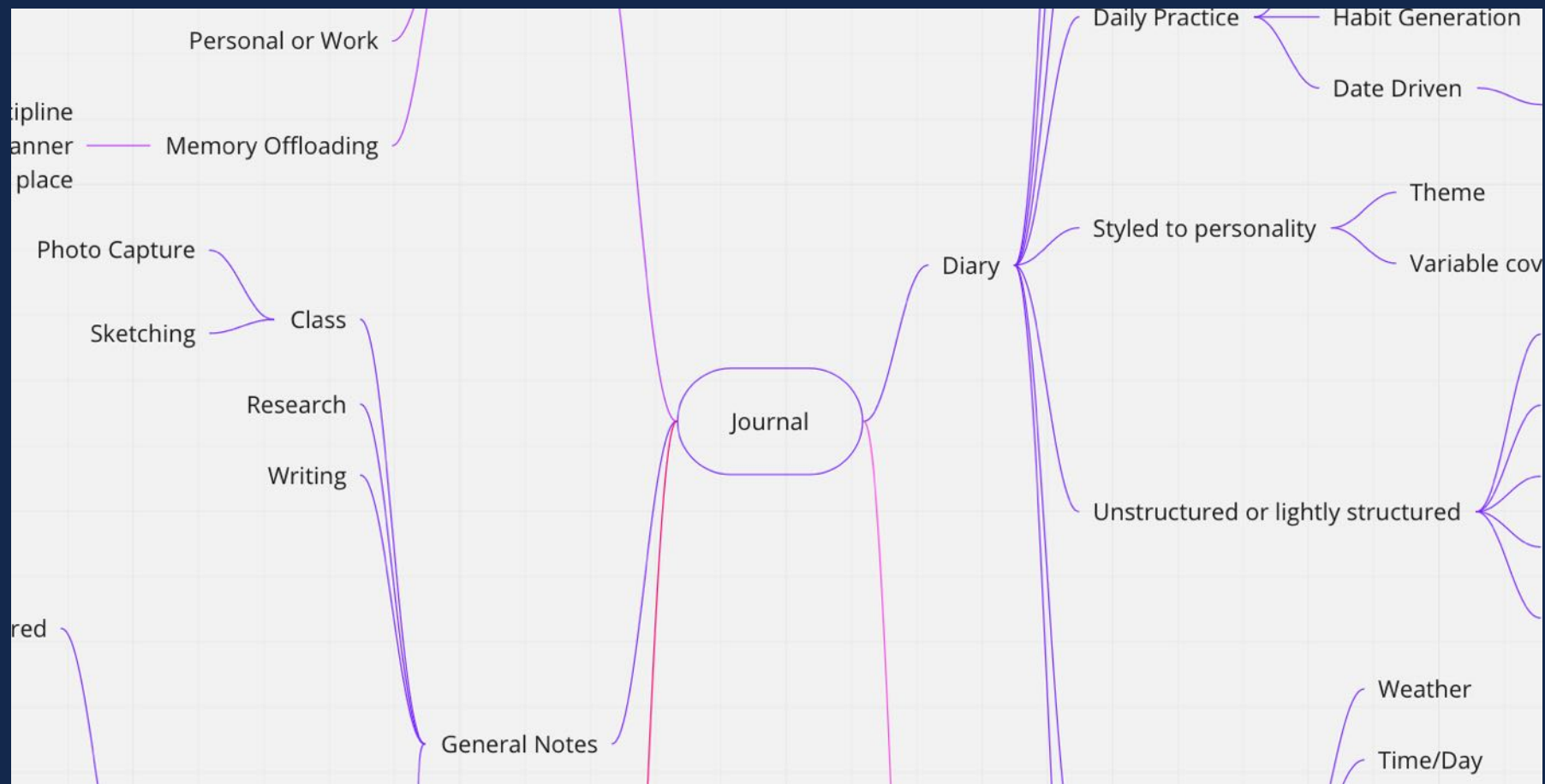
Technique: 5W1H or Sometimes 5W2H with 2nd H = How Much

Who	→	• Address Book (deprecated) • Maybe Journal about your kids?
What	→	• Hobby journals • Maybe Health
When	→	• Monthly / Annual Journals
Where	→	• Trip / Vacation Journal
Why	→	• Confession Diary?
How	→	• Instructional Guide Journal

Thinking About the Domain



Thinking About the Domain





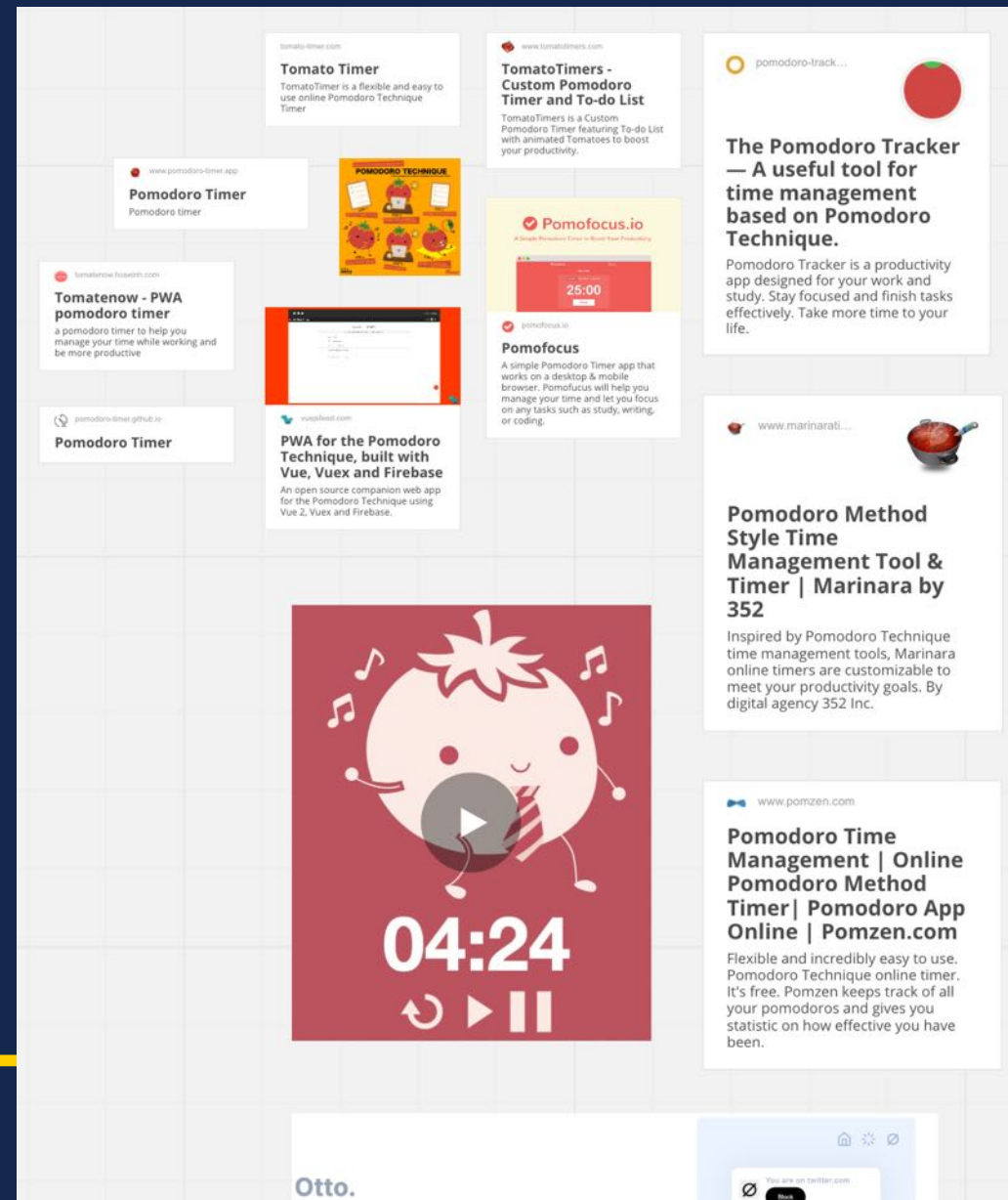
Take 2 - Pomos

Ideation Example - Pomodoro Timer Pt. 1

Key Takeaways:

- Guilt-tripping people from stopping their timer early
 - Don't kill the tree
 - Using discouraging language like "Give Up"
 - Having a doodle of a bear that's angry at you
 - Literally charging you money
- Social Networking
 - Creating group trees you build together
 - Competitive ranking
 - Daily feed of your friends' Pomo activity
- Gamification
 - Sometimes a complete game; rewards/currency for completing Pomos, things you can buy with what you win
 - Badges you can unlock
- Study Techniques
 - Most are "focus" based
 - Flashcards
- Aesthetics
 - Most every app tends towards a super thin font for the time, hard to read
- Paid vs. Free
 - Almost no correlation between whether I liked the app vs. whether it was paid
 - Apps with a free version almost never justified the upgrade to the paid version with enough features

Should you consider that it is just timers and be more general?



Ideation Example - Pomodoro Timer Pt. 2

What is the Pomodoro Technique?

- The Pomodoro Technique is a time management method developed by Francesco Cirillo in the late 1980s.
- Using a kitchen timer shaped like a pomodoro (Italian for tomato), Cirillo divided the time he spent working on a project into 25-minute intervals, with 5-minute breaks between, in order to get more done, without interruptions.
- By grouping a number of Pomodoros together, users can tackle a project of any length and drastically improve their productivity, enhance their focus, and achieve their goals.
- With this tried-and-tested method, users can simplify their work, find out how much time and effort a task really requires, and strengthen their focus so they can get more done in the same amount of time each day.

Goals of the Pomodoro Technique

- The aim of the Pomodoro Technique is to provide a simple tool for improving productivity.
- It can do the following:
 - Boost motivation and keep it constant
 - Enhance focus and concentration by cutting down on interruptions
 - Refine the estimation process in both qualitative and quantitative terms
 - Increase awareness on one's decisions

From: <https://www.pomzen.com/>

Pomodoro Encouragement vs. Discouragement

Discouragement of Quitting via Penalty (Stick)
Encouragement of Finishing via Reward (Carrot)

Stick Methods

- Donate to a negative cause
- Cost you money
- Social Shame (expose your failings to others)
- Tug on heartstrings (death of your cyber creature)
- Judgement (App chides you)
- Ranking / Competition

Carrot Methods

- Badges
- Level Up
- Virtual Currency
- Upgrades for sounds, cyber pet changes, etc.
- Ranking / Competition

Consider - The Medium

Thinking About the Medium

- Your medium is a web application, but more than that:
 - What device is it intended for, mobile or desktop?
 - Where is your application likely to be used? In a kitchen? In a restaurant? At a desk?
 - Is there a physical equivalent to your app? What do people like about the physical version?
 - Have other people attempted to do what you're doing? What do their apps look like? What are the reviews saying about their apps? What works and what doesn't?

Thinking About the Medium Cont'd

Physicality	
Why use a physical notebook	Possible Digital Implementations
• Feel the texture of the binding / pages	• Allow users to pick through various cover and page textures
• Open and close it	• Have users do a gesture to open / close the book to write in it
• Handwrite slowly	• Let user's pick from various handwritten fonts / ink types for those fonts
• Complete the book	• Create a page limit on the notebooks, allow users to select small / medium / large
• View all previous completed books on shelf	• When selecting your current notebook, pull it from shelf of all completed notebooks / maybe home screen could be that shelf
• Easily Doodle	• Have a doodle mode that clears away everything and let's you scribble to your heart's content
• Confidence in ownership	• Have strict offline mode option

Thinking About the Medium Cont'd

- What are the advantages of using a web application over a physical product in this case? What are the disadvantages?
- Can these disadvantages be overcome? Or should this application be designed in a way such that those disadvantages are no longer relevant?

Thinking About the Medium Cont'd

Physical Journal Product Reviews

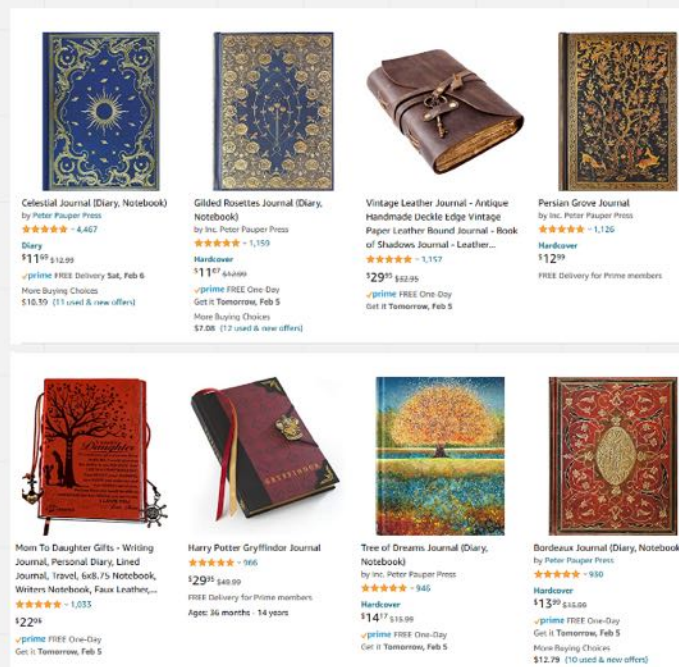
https://www.amazon.com/Celestial-Journal-Diary-Notebook-Pauper/dp/1441316698/ref=sr_1_1?dchild=1&keywords=journal&qid=1612478750&sr=8-1#customerReviews



Celestial Journal (Diary, Notebook) - **\$11.69**
4.9 / 5 Start - 4,467 reviews

Top Comments in Reviews:

- Reproduction of old Design
- Design isn't flat, wraps around, reflective gold
- Small size (~6"x8") with nice number of pages (160)
- The design makes it cool for recording what you see watching the sky
- Commenters are hesitant to write in it bc it's so nice



It seems that the most popular journals have to do with aesthetic much more than over specific function. The vintage, old, higher quality aesthetic seems to be a persistent theme. People like the feeling of quality: Thick, heavy duty, high quality materials (leather, hard bound, etc), heavy weight, things like that

Thinking About the Medium Cont'd

- When we were making a bullet journal application, we wanted to translate the quality of the physical-ness to digital
 - The advantage a really nice journal has over an app is the texture and materials, it feels nice in the hand, there's quality to it that makes just holding and writing in it extravagant
- We researched different ways to bring texture and high quality materials to a digital space

Thinking About the Medium Cont'd

- The way the user interacts with your application is almost the most important part of it.
- To the end user, UI / UX trumps *everything* cool or neat that you did in the backend
 - This is where the bulk of your time should be spent. Users are much more willingly to overlook some minor functionality quirks if what they're looking at is pretty
 - Look at apple: plenty of bugs to be found, but people generally associate it as a better experience than android because it is aesthetically nicer

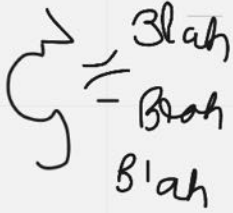
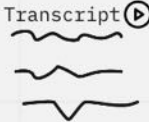
Thinking About the Medium Cont'd

- Typing on a mobile device / tablet will never be that great of an experience, especially when compared to hand writing a note
- How can you remedy this? Drawing with a stylus if the (device has one)?
 - Drawing on a screen is ok, but often doesn't look as pretty as we wish it did, that's the one upside of typing
 - What's the compromise?

Thinking About the Medium Cont'd

- Typing on a mobile device / tablet will never be that great of an experience, especially when compared to hand writing a note
- How can you remedy this? Drawing with a stylus if the (device has one)?
 - Drawing on a screen is ok, but often doesn't look as pretty as we wish it did, that's the one upside of typing
 - What's the compromise?

Thinking About the Medium Cont'd

People want to have the journal in their own pen				
People want their journal to look prestine and only "wabi-sabi" style messy				
Typing is not nearly as engaged though it is more efficient than writing				
		<u>Capture</u>	<u>Rendered</u>	<u>Stored</u>
	WRITTEN	<i>Jest</i> ←→ Test	Test	"Test"
		* ←→ *	* * * * *	important: true
Crazy Idea Ask the user to train or make good symbols of -, *, ! and so on Then if they type them we replace them with that?				
Could we have you write them and like the auto-detect it it turns it into their writing in a nice style				
Think of this as a "Smart and Personal Handwriting Improvement"		AUDIO		 032721.mp4
We should have a toggle that the user can type and then it can flip into writing in case we are capturing a large amount of content. The reverse of course is that we are able to flip back to the typed version				

Consider - The Users

Thinking About the Users Practically Speaking

- Friction

- How much effort is it to use your app? Is there a long setup process? Are they going to have to sort through hundreds / thousands of recipes to find one they might like?
- For the journaling app we realized that when daily journaling sometimes you just don't have the motivation to write a long journal entry, or answer a lot of prompts.
 - This is *high friction*
 - To lower the friction, we thought of a special low effort “quick note” button with one or two short prompts just to minimize the number of “zero” days where nothing is written

Thinking About the Users - Friction

Reducing the Friction / Burnout

Some days you just aren't going to feel like journaling - and that's ok

How do we adapt to this, so that the user can easily log *something* and not feel as bad for missing a day?

Have a "low effort" mode (working name, should probably come up with something better

Low Effort:

- Only asks questions instead of being open ended writing
- Gives limited amount of responses to reduce decision fatigue
- Short - Can be completed under a minute
- Prompted at the end of the day if it is a bit past when your normally scheduled time is and you haven't completed it yet

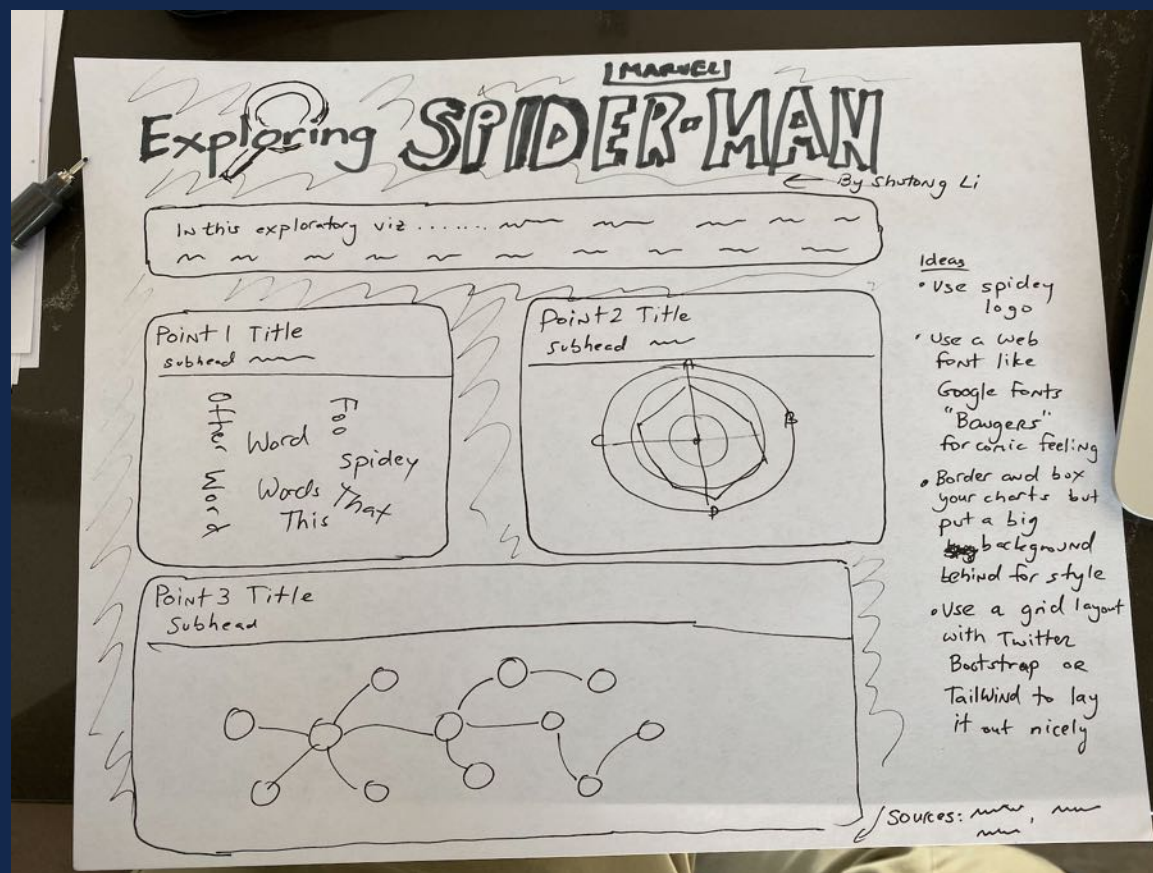
Normal Effort:

- Prompted at regular time
- May or may not have questions associated with it depending on if you had a template / what kind of template you have
- No estimated time length
- Possibility of being more free form depending on template
- This should be the default and assumed effort

Visualize Your Idea!

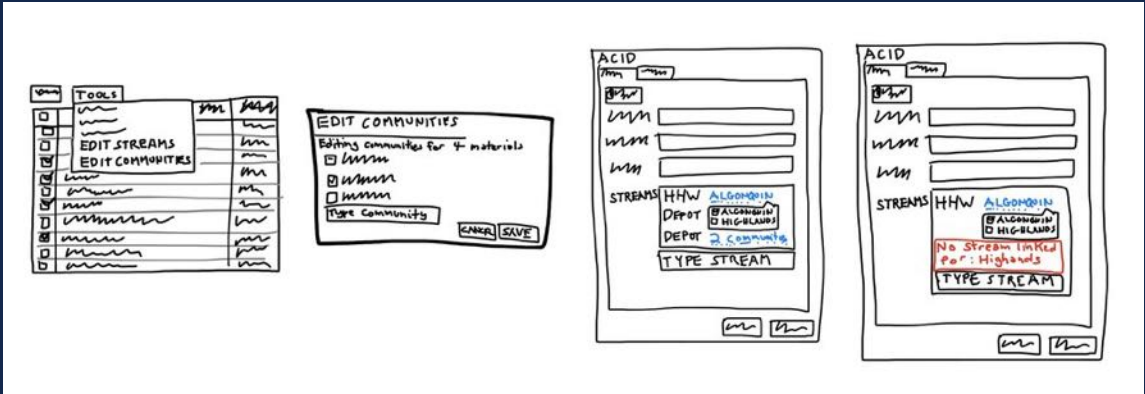
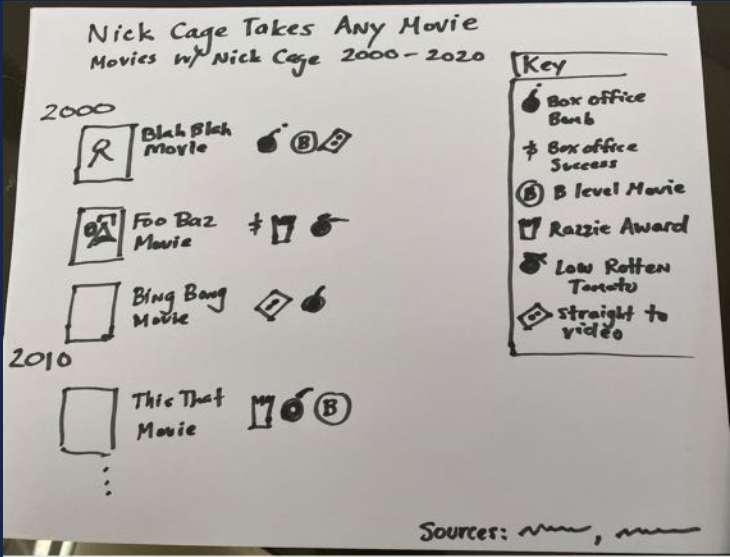
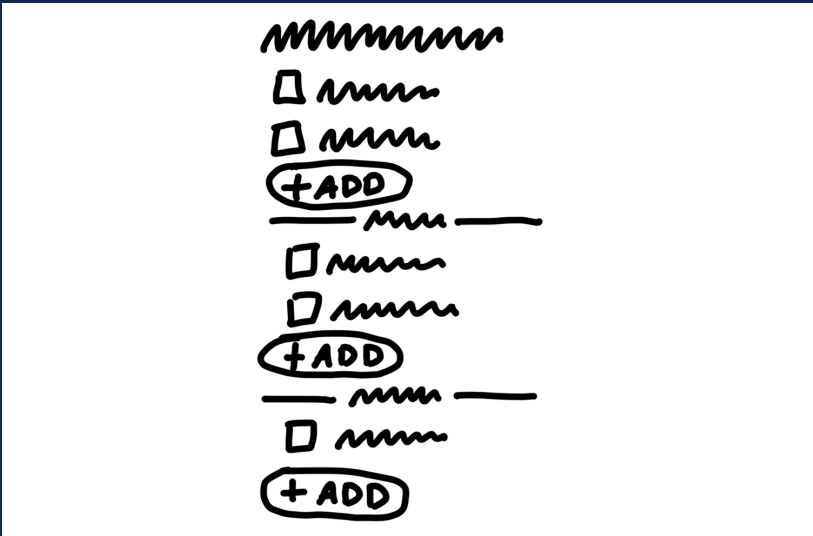
Technique - Fat Marker Sketching

- Work out your ideas very roughly at first.
- Go fast, let things happen, use physical medium or specialized ideating tools to avoid trying to getting to finished directly
- This can happen on paper or in a tool like Miro

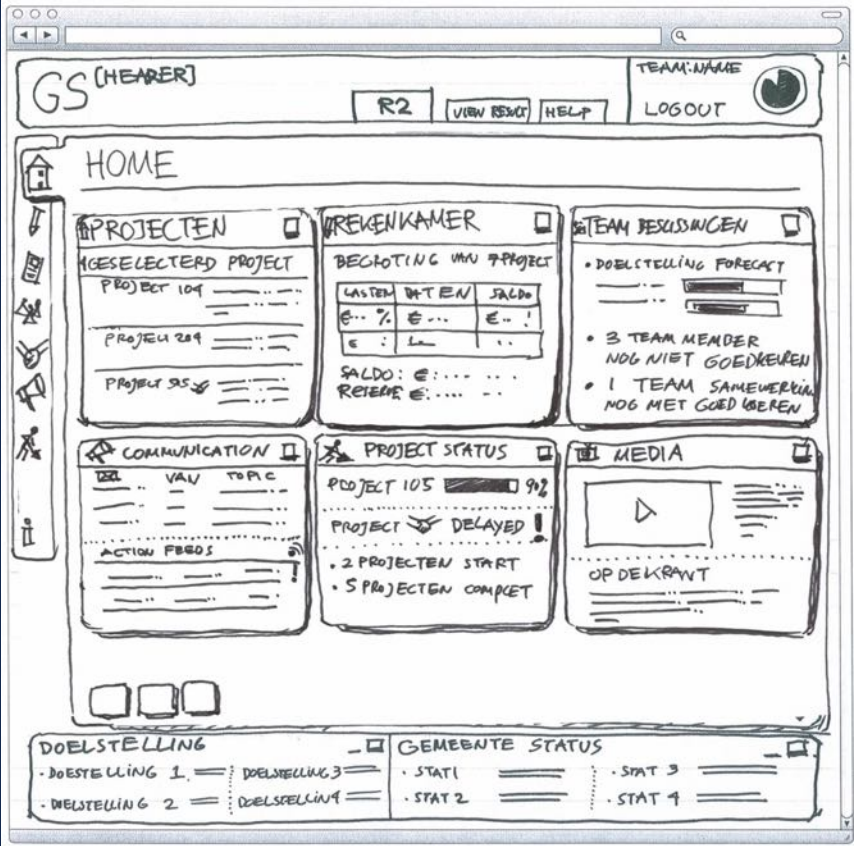
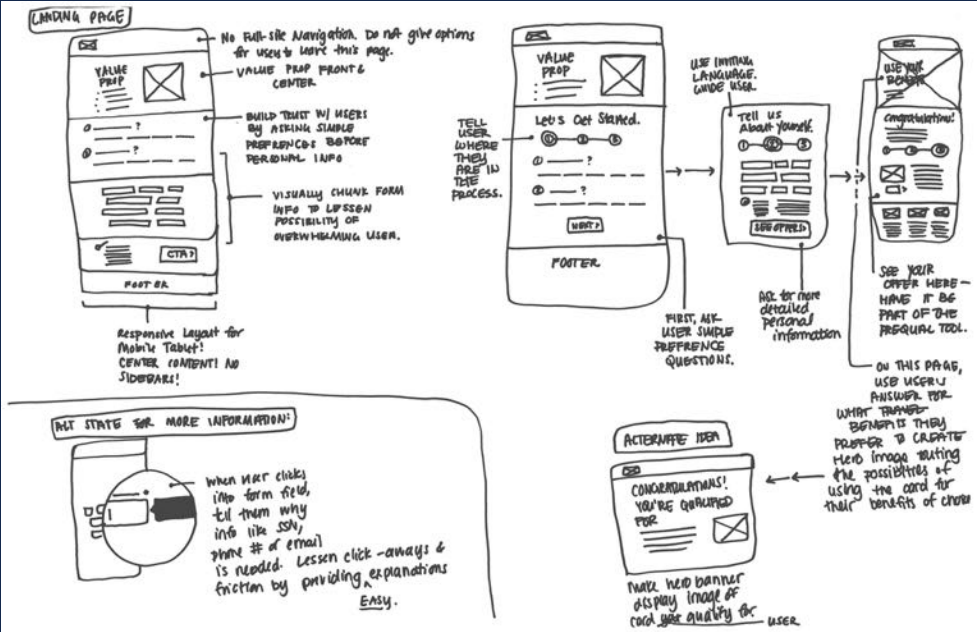


Me helping a student with a MCU dashboard after ~5mins

Fat Marker Sketching Fidelity Examples



Careful You Don't Start Going Too Far

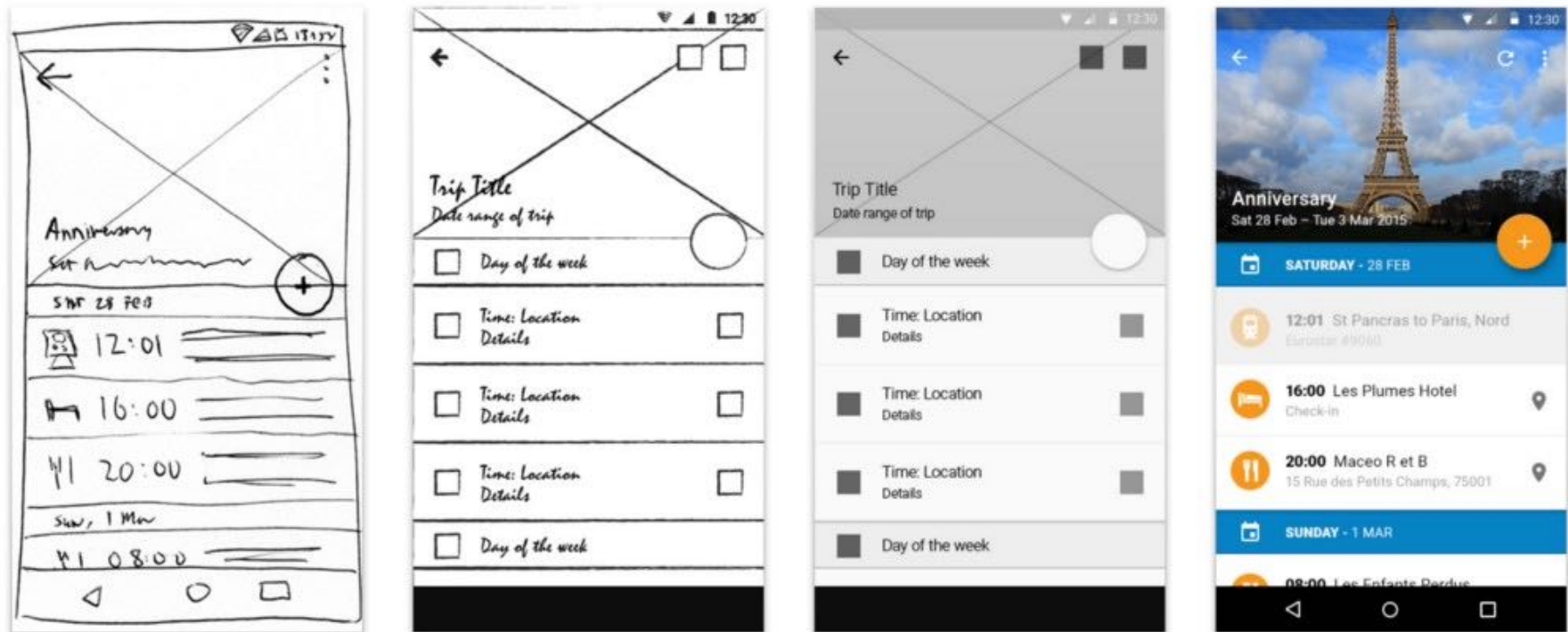


Are you getting into design and specifications maybe too early?

Eventually You Take the Road from Low Fidelity to High Fidelity

- Your prototype should start low fidelity to work out basic issues (fat marker, rough sketches, etc.)
- Next you should refine the idea with wireframes and sweat the details
- Finally you should explore the pixels and get the look and feel right

Low-Fi, Med-Fi, Hi-Fi



Source: <http://murdochcarpenter.com/portfolio/wireframes/>

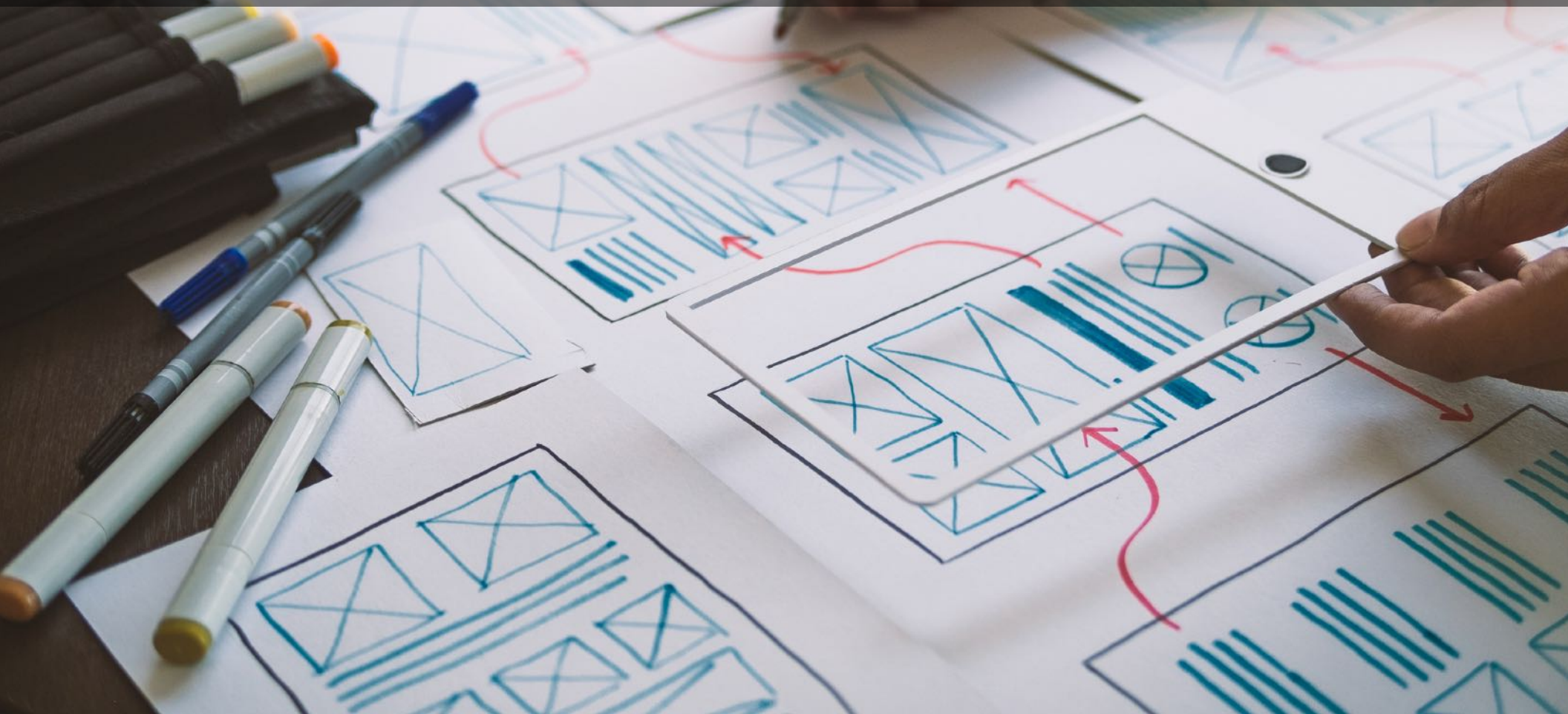
Wire Framing

Work Out General Features and Organization



Sequenced Wireframes

General Flow



Storyboarding

A Bit More Fidelity and Interface Details



Design Comp Creation

High Fidelity Design and Detail Exploration



Prototype Tips

- Build your prototype in the most time and fidelity appropriate way.
Early tends to be fast and ugly and later tends to be slower and fancier.
- Test your prototype in context
 - On phone, on watch, on screen, etc.
- Goldilocks prototyping - not too little and not too much
 - Should be proportionate to implementation difficult and acceptance concerns or more globally RISK

Risk - Early pixels

- Be very careful in showing pixels very early
- Since the interface is the application to users a very rich prototype sets the expectation you are almost done when you very well might not be (ex. Complex implementation)



Accept the Mess and Plan For It?



AnaB @NotechAna · 3h

...

Replying to [@johncutlefish](#)

There's a principle in urban planning, where architects follow "free-will paths" - the shortcuts that people make to get where they want. Wonder if there's a way to implement smth like this in planning data pipelines



4



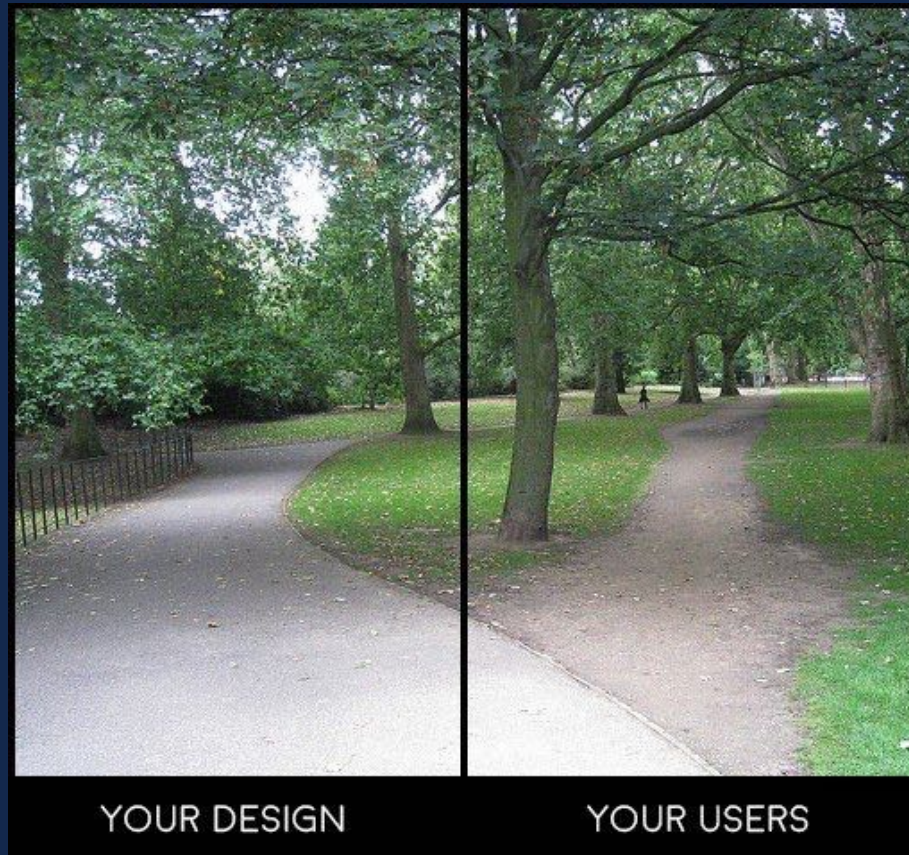
Mark Vasko @MarkVasko · 2h

...

Replying to [@johncutlefish](#)

But that means we'd have to acknowledge that these systems are messy, complex, adaptive, and why would we want to acknowledge that?!

Risk - Think for Actual Use, Not Just Intended Use



Responsible devs must ponder early on what users will do with their creations and how things might be misused purposefully or by accident.

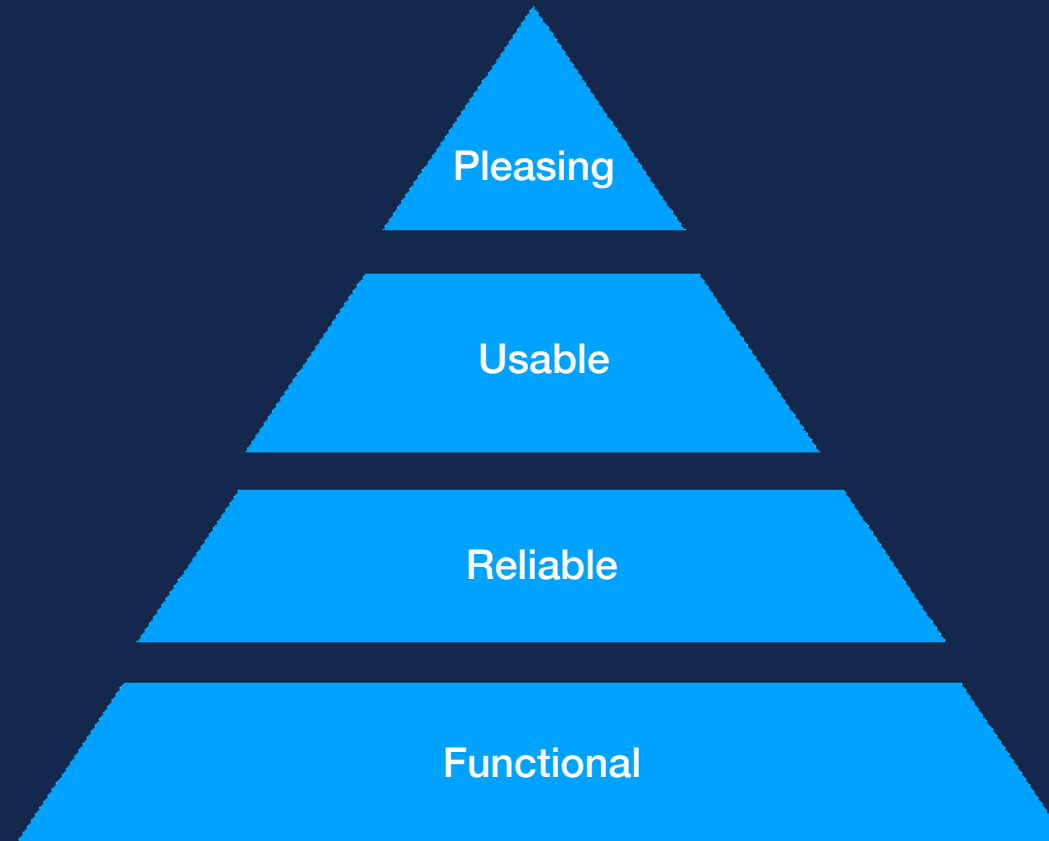


The overview **effect** is a
cognitive shift in awareness.

**Does our industry
need an overview
effect to see our
impact?**

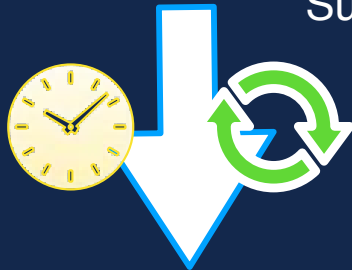
Statement of Obviousness
Online World == Real World

Idea - Scale Maslow's UX Pyramid ?

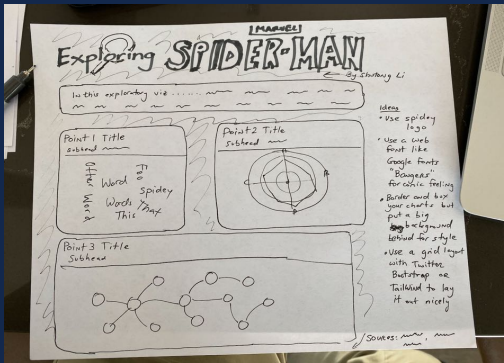


Ok getting a bit more
focused

Formula 1.0 for the Project Brainstorm

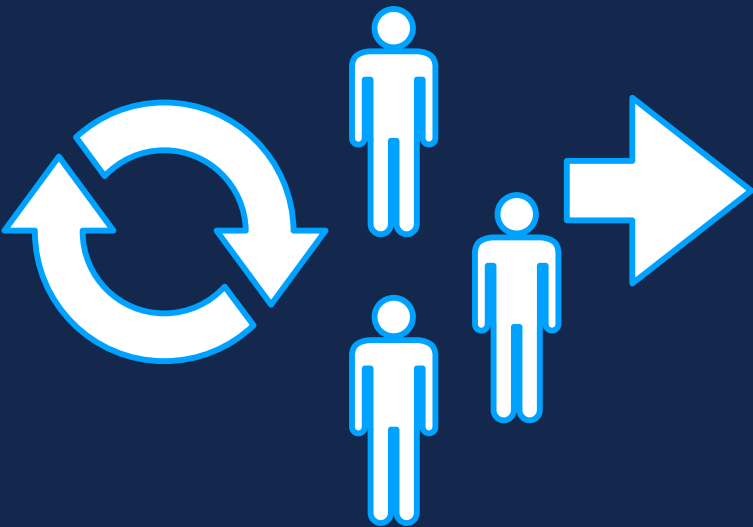
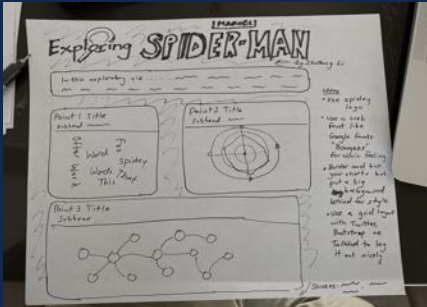


Finished Rough
Concept for Specifying

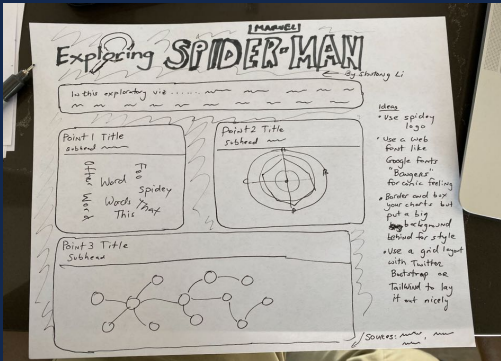


Add More Feedback to Formula

Initial Concept



Finalized Concept



Feedback

Stake Holders

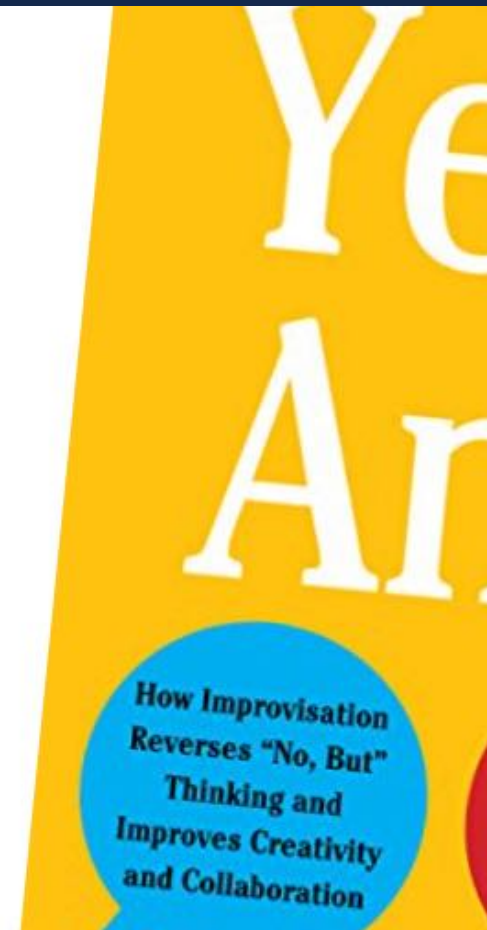
End Users

Brainstorming Tip: Support a Yes, And... Mood

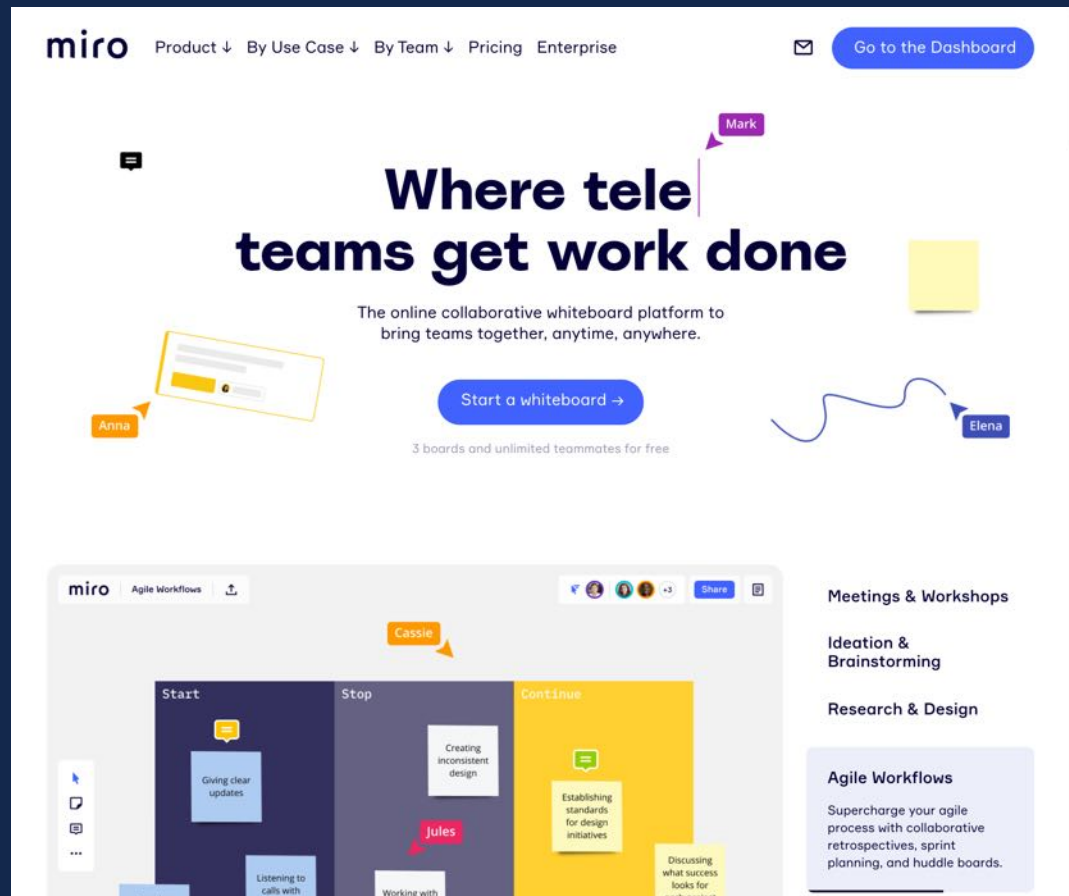
Professional success often rests on the same pillars that form the foundation of great comedy improv: Creativity, Communication, and Collaboration.

Yes, And

Kelly Leonard, Tom Yorton



Brainstorm Tooling: Demo Time



<https://miro.com>

Summary

- Gathering requirements is required before you make a plan
- You will need to balance the needs of stake holders and end users
 - Be very careful in your role here as it may significant consequences
- Systems and User Centered Thinking is paramount to good early requirements planning
- Enough time should be given to brainstorming and idea gathering so you don't build the wrong thing